ReSearchOS：面向持续科研的人机协作参考架构

技术报告 · 未经同行评审

Shiyu Hu*
新加坡南洋理工大学物理与数学科学学院
shiyu.hu@ntu.edu.sg

2026 年 10 月 29 日

当执行越来越容易被委托，研究者更需要决定：
什么值得追问，什么足以成为根据，以及什么值得留下。

摘要

在跨越许多次 AI 辅助会话的研究里，一篇研究者依赖的论文在几周后被更正。这时要做三件事：（1）更正没有触及的判断，连同根据一起保留；（2）被触及的部分要更新，而且只更新那一部分；（3）证据还不能裁定的问题要挂起，等研究者来回答。哪部分保留、更新或挂起，应由研究者决定，而不是模型或另一个会话：要留住的是研究状态，不是研究文件。ReSearchOS 就是按这个思路建的环境，由研究者拥有，以本地文件为主。模型的产出一直只是候选，研究者准入之后才进入研究状态，也就是问题、证据、带范围的已接受判断、未决条件和下一步。本报告给出三部分内容：规定这样的环境必须保住什么的设计原则，让保留、更新与挂起可以检查的研究状态模型，以及参考架构，附命令行原型和一个可以安装、只复制的公开 starter。原型在下一条命令时发现来源变化，为依赖它的判断登记复核，并拒绝审阅期间来源已变的候选。在贯穿会话里，研究者把收窄的判断作为新块准入在原判断旁边，并拒绝了一个没有依据的提议。原型目前只实现了一部分；使用研究和与「强模型加一页交接」的比较是下一步。

关键词：人机协作；研究状态维护；参考架构；判断连续性；可扩展性。

1 研究状态，而不是研究文件

跨越许多次 AI 辅助会话的研究，每一步都建立在早先的判断之上，而判断依据的来源会随时间变化。所以只把眼下这一步做对还不够，从前面带下来的东西是否仍然成立，也要重新核查。那么来源变了，研究环境该做什么？先看一个例子。

一位研究者想知道两种方法 A 和 B 哪个更好。找到的一篇论文做了两组基准测试，两组里都是 A 更好，于是记下结论：A 优于 B。不过论文只测了一种情形，换一种情形 A 是否仍然更好，论文没有说，只能留待以后自己做实验。

几周后论文被更正：第二组测试的数字算错了，改过来以后，A 在这一组上的优势消失了。同一周，研究者的学生让 AI 助手在另一个会话里读了更正后的论文，AI 助手说：在没测过的那种情形下，A 也会更好。

*本文介绍作者独立设计与实现的个人项目 ReSearchOS 的设计原则、研究状态模型与命令行原型。本文未经同行评审，原型只实现了一部分；其中的形式化内容是为了把设计说清楚、便于讨论，不构成经过评审的结论。本文用于理解与交流。

结论该怎么处理？第一组测试没有错，它对结论的支持照旧，要**保留**（keep）。第二组变了，结论里靠它的那部分要**更新**（update），别的不动。没测过的那种情形还没有答案，继续**挂起**（hold），留给计划中的实验；AI 助手的说法在论文里找不到依据，不能收下。保留、更新与挂起，问的其实是同一个问题：新证据碰到了什么。

这些事为什么不能交给工具自动去做？在构建系统里，输出是输入的函数：输入一变，依赖它的部分就重建，不必问任何人（图 1a）。判断却不是来源的函数。同一次更正，可以让判断的一部分照旧成立，让另一部分需要改，还留下一部分无法裁定；哪部分归哪一种，要由读懂证据的人来判断（图 1b）。模型和别的会话可以提出看法，学生的 AI 助手那条提议就是例子，但它们和文件都不该替研究者作这些决定。系统能做的，是找出哪些判断依赖变了的来源、把它们交还研究者，未经研究者决定，不把任何东西准入长期状态（图 1c）。

后文把这条结论记作 J。例子里的两种方法是目标跟踪方法，即在视频里一直跟住某个目标。论文只测了目标始终留在画面内的情形，所以 J 完整地说是「目标在画面内时，A 优于 B」。没测过的，是目标离开画面又回来的情形。报告把这两种情形分别称为画面内条件和重新出现条件。

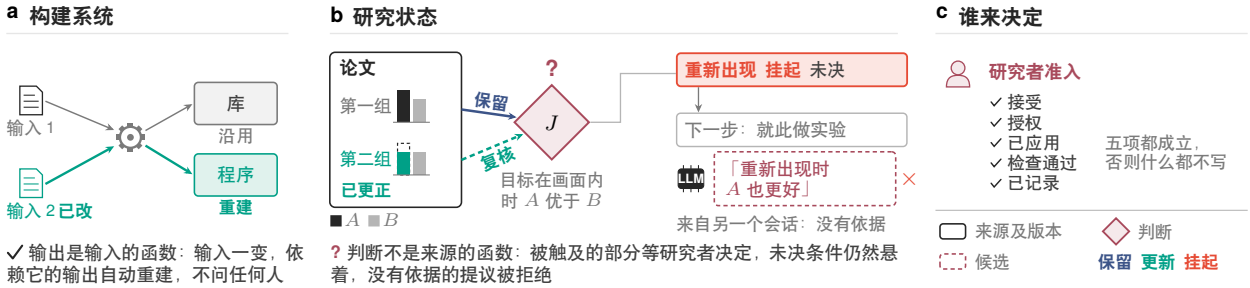


图 1. 构建系统自己重建，研究状态等研究者决定。 **a**，构建系统里输出是输入的函数：某个输入一变，依赖它的程序被重建，库照旧沿用，不问任何人。**b**，研究状态里判断不是来源的函数。论文的第二组基准被更正后，J 来自第一组的根据保留，来自第二组的根据要复核，所以 J 等研究者决定。未决的重新出现条件仍然悬着，另一个会话提出、论文并不支持的说法被拒绝。柱子只是示意形状，不是数据：更正后，A 在第二组上不再领先。**c**，只有研究者的准入才写入。全文沿用这套记号：方框是来源及其版本，菱形是判断，虚线框是候选；蓝、绿、橙红分别表示保留、更新、挂起。

这样一次变化之后要留住的，正是研究状态（research state）：问题、证据及其关系、带范围的已接受判断、未决条件，以及有依据的下一步。在例子里，它们依次是在 A 与 B 之间的选择、论文的两组基准、适用于画面内的判断 J、未决的重新出现条件，以及计划中的实验。笔记若只写「用 A」，这些都会丢失，连未决条件和接下来的实验也不例外。本报告的回答由此而来：要留住的是研究状态，而不是研究文件；来源发生变化时，哪部分保留、更新或挂起，由研究者决定，而非模型或另一个会话。[1]

研究状态会以三种方式变化，例子里三种都有。时间在推移：论文被更正，会话结束，其间用的工具和模型也可能换掉。用途转向目标重新出现的情形，条件就变了。另一个会话送来提议、得有人决定什么算正确，参与者也就变了。研究者已在用的工具也能保留、更新记录，有的还会把变化了的来源、或建立在它上面的主张提示给人。但第 5.1 节对照的几类工具里，没有一类按范围与依据版本保存判断，并只把变化触及的部分交回研究者决定；该节把同一次更正逐一放进这些工具里对照。

ReSearchOS 就是为来源变化的这一时刻建的环境，由研究者拥有、以本地为主。前面说系统能做的最后一件事，未经研究者决定就不把任何东西准入长期状态，在这里成了它唯一的权威规则：机器的产出一律先作为候选（candidate），即尚未准入的产出，只有研究者能把候选准入研究状态。准入（admission）是把候选变为长期状态的一步，须经这个决定，并留下回执（receipt），即做了什么、查了什么的记录（第 3.1 节）。

[1] 作者在别的工作里研究过类似的问题，对象是跟踪系统和学习者。本报告只把这个问题用到科研实践上，并不把这个环境当作那些工作的成果。

ReSearchOS 的名字取自 Re Search：带着新问题重返材料，并保留这次考察带来的变化。[2]研究者与计算辅助系统在其中围绕共享研究对象，跨会话形成问题、审查证据并修订判断。它所服务的设计要求，我们称为判断连续性（judgment continuity）：恢复早期工作的根据，并重新判断其是否仍然适用。这个环境保管共享记录与未完成的工作，工作本身则在 Agent 会话、命令行或专业工具中进行，环境并不编排这些工具。

对研究者来说，这条规则落在一个闭环里：恢复此刻要紧的状态，在实际开展工作的客户端中推进研究，只确认值得进入长期状态的变化。客户端（client）指开展工作的任何入口，例如命令行、Dashboard 或 Agent 会话，所有客户端面对的是同一套记录。恢复只为手头的工作选取最少的上下文，而非整个仓库。确认则问这次会话产生了什么值得未来继承的真实变化，没有就不写回。第 4.1 节把这个闭环展示为一次会话中的六条命令。

1.1 三项贡献

第一项贡献是设计原则（第 2 节），规定环境必须保住什么，保留、更新还是挂起才始终由研究者决定：人的判断、带条件的组合、比工具活得久的记录、可纠正的积累。在例子中，原则要求未决的重新出现条件始终与 *J* 记在一起，学生的 AI 助手所作的提议也只有经研究者接受才能成为状态。

第二项是模型（第 3 节），它让原则可以检查。模型讲明研究状态，以及保留、更新与挂起各自对维护工作的要求，三者各占一个小节（3.1–3.3），再在有界目标上把三者组合起来；有界目标是带冻结范围与明确结果的维护工作单元（3.4）。在例子中，模型说明 *J* 何时可以支持重新出现实验，论文的更正对这种支持有何影响，以及工作何时算作完成。

第三项是参考架构与原型（第 4 节），它们把模型落到研究者实际运行的命令上，在命令行上承载变化发现、复核和研究者的决定。在例子中，原型恢复这次比较，论文一经更正就把 *J* 标为待复核。在终端上，它只凭研究者的决定准入收窄的判断，而且作为 *J* 旁边的新判断块，不改动 *J* 本身。审阅期间来源变了的候选，它会拒绝；他人的修改，它也不会覆盖。报告还给出一个可以安装的公开 starter，默认只复制：回收候选，但从不准入。

> **本报告怎么读**。初读可读本节及其术语表（表 1）、表 2 中的原则和第 4.1 节的贯穿会话。正在考虑要不要试用的读者，请先读第 5.2 节，那里陈述了用「强模型加一页交接」代替本环境的理由；如果交接胜出，ReSearchOS 应收缩为最小形态。第 3 节借贯穿例子讲解模型。附录 A 列出模型的对象和要求的生命周期，补充说明 5 给出每条定义的精确表述，并论证两条命题。附录 B 是实现记录，补充说明 6 列出全部差距。

表 1 给出初读者在第 3 节之前会遇到的词及其平常含义。模型自身的术语，如要求、检查、挂起与等待，在第 3 节首次用到时结合例子讲解；附录 A 的表 8 连同状态列出它们，补充说明 5 给出精确定义。多数名称与原型的代码和命令一致；代码把有界目标称为 episode。

[2]另有公开项目也使用 ResearchOS 这一名称，如 `github.com/CognitiveAISystems/ResearcherOS` 与 `github.com/JRH89/researchos`；ReSearchOS 与它们无关。

表 1. 全文用到的术语

术语	在本报告中的含义
研究状态	研究要带下去的东西：问题、证据及其关系、带范围的已接受判断、未决条件与下一步（第 3.1 节）
客户端	开展工作的任何入口，例如命令行、Dashboard 或 Agent 会话；所有客户端面对的是同一套记录
卡片	一份 Markdown 记录，例如承载 J 的那一份
候选	尚未准入的模型产出或草稿
准入	候选进入研究状态的一步；它需要研究者的决定（第 3.1 节）
回执	做了什么、查了什么的记录
有界目标	带冻结范围、冻结规格和明确结果的维护工作单元；原型里由一次会话近似代替它（第 3.4 节）
公开 starter	读者能够安装的版本，默认只复制：回收候选，但从不准入。私有版在此之上加入准入与 Dashboard 写回

主张边界。第 3 节的模型是规格，两条命题只在与之一起陈述的假设下成立，假设见补充说明 5。第 4 节描述的原型只实现了一部分，对应的提交列在附录 B.3，贯穿会话展示的就是这些提交做到的事。原型只在终端上以研究者的决定为准（差距 1、2），其余尚未做到的列在第 4.5 节。到目前为止，原型只有作者一位用户；原型由作者开发，本文报告的每一项结果也由作者评价，因此没有一项独立于作者。使用研究还没有做，与「强模型加一页交接」的比较也还没有做，所以本文不确立真人验收或科研收益。这个环境面向已有来源核查习惯、需要跨会话工作的研究者；新手怎样借它学习、团队怎样在其中处理分歧，需要另外研究。

2 研究环境必须保住什么

要让第 1 节那个决定始终留在研究者手里，研究环境必须保住什么？设计的出发点是一句话：把研究分解到能作判断，再组合回可以继续的状态。状态一旦变化，这句话要的正是保留、更新与挂起：哪些照旧成立，哪些要修订，哪些要等回答。

四项原则各防这三个动作的一种出错，顺序跟着一条判断的经历走：作出、复用、跨工具保存、纠正。模型的产出没人决定接受就成了状态，这是保留了从未被接受的东西（第 2.1 节）。后一步接着建在一个从未确立的结果上，这是在未知上更新（第 2.2 节）。记录只有一种工具读得了，那种工具一旦不在，本该保留的东西也跟着丢了（第 2.3 节）。纠正靠抹掉历史而不是收窄主张，这是用删除来更新（第 2.4 节）。效率、控制与长期留存相互拉扯时，原则优先保证四件事：研究者的重要判断、带条件的组合、比工具活得久的记录、可纠正的积累。图 2 画出从原则经模型、架构一路推到代码的过程。

这些原则不规定统一的目录、方法、界面或审阅安排。解释要花时间读、花力气维护，审阅也可能打断工作，或者久而久之流于习惯，所以检查集中在要紧的转变处，常规工作就在约定的范围内照常进行。

四项原则来自作者自己的设计史：这段历史第一天记下的第一个请求，就已在理解研究判断与模仿措辞之间划了界线。规则本身写在作者的六份内部设计文件里，每条规则只由一处负责，并由一份冻结的合同与检验它的测试连在一起；本节的概括已对照这些文件核对过。表 2 列出每项原

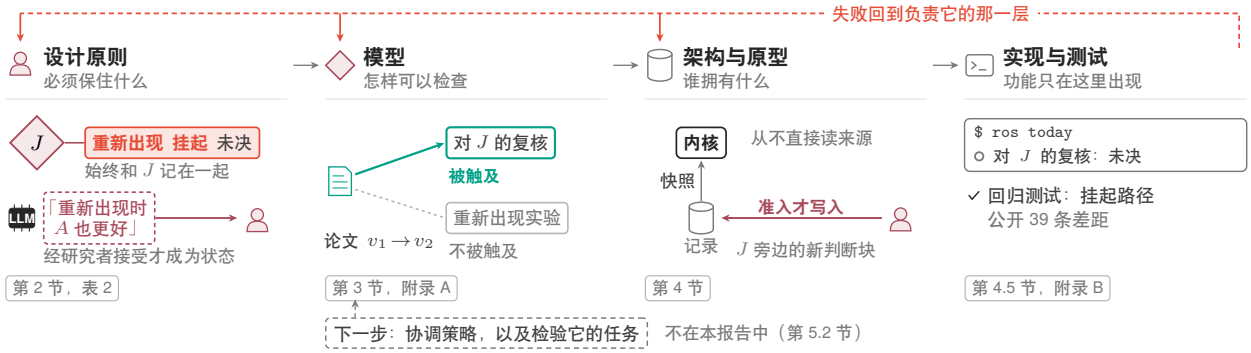


图 2. 原则、模型、架构与实现各回答一个问题，失败回到负责它的那一层。每一层都画出它对贯穿例子说了什么。设计原则说明必须保住什么：未决的重新出现条件始终和 J 记在一起，学生的提议只有经研究者接受才成为状态；它定下人的判断、带条件的组合、比工具活得久的记录和可纠正的积累。模型让它可以检查：论文从版本 v_1 变到 v_2，对 J 的复核被触及，重新出现实验不被触及；它定下研究状态、变化触及什么、三值检查、挂起与等待以及有界目标。架构与原型说明谁拥有什么：内核从快照计算，从不直接读来源，判断只经准入写入，写成 J 旁边的新判断块。单一内核是目标（差距 3）；它定下一条准入链和不另建事实存储的适配器。功能只在实现与测试中出现：`ros today` 把对 J 的复核记为未决工作，回归测试检验挂起路径，39 条差距公开列出。灰色标签是各层在本报告中的位置。协调策略及检验它的任务是下一步，不在本报告中。

则回答的问题、规定与机制。补充说明 1 给出各项原则的例子与来源，补充说明 3 完整复述这些条款，并注明每条由谁负责。

表 2. 四项设计原则及其机制落点

原则	它回答的问题	规定	机制落点
人的判断	由谁接受判断？	只有研究者决定什么成为长期状态；机器的产出一律先是候选	候选、决定、修改、检查与回执分别记录
带条件的组合	结果在什么条件下可复用？	后一步只在前一步的实际结果确立其前提时才开始；未知不算满足	带声明前提与检查的规则应用、组合（第 3.4 节）与三值检查
比工具活得久的记录	更换工具后保留什么？	本地可读、可版本化的来源为准；模型、客户端、插件可替换	来源归属、上下文与外发回执、依据钉在来源版本上
可纠正的积累	纠错后哪些判断需要改变？	纠正只收窄受影响的主张，不抹掉历史；失败、未知与未验证如实保留	判断修订、回滚回执、能力证据跨会话累积

2.1 只有研究者能把候选变成状态

第一种出错是保留从未被接受的东西。在例子中，学生的 AI 助手提出 A 在目标重新出现后也更好；这条提议若未经研究者决定就被准入，J 就会扩展到论文并不支持的情形。所以只有研究者决定什么成为长期状态。模型依据能看到的材料作出解释、提出候选；研究者判断证据是否充分、哪些问题值得追、何时转向。接受与否由研究者决定，而不是由提出它的模型决定，所以长期保留推荐、扩大范围或把材料交给新的接收者之前，都要先经审阅。承载这项原则的是准入：准入的五个条件全部成立，候选才能进入研究状态，第一个条件就是研究者的决定（第 3.1 节）。

这项原则借鉴的研究中，有三类值得在这里说。认知强制方面的研究 [3] 表明，不加选择地接受，容易导致过度依赖。Clark 与 Brennan 关于共同基础（grounding）的论述 [5] 认为，理解到什么程度算够，取决于当前目的需要什么，而不是反复确认。QOC 记法 [1] 记下问题、选项与标准，这里的决定也一样，连同根据与范围一起保留：比如选方法时，要留下的是为什么可复现性比报告里那点微小的精度差异更重要。准入确定的是谁接受了候选；候选对不对，仍要靠后面的检查，回答一致并不证明科学判断正确；修订判断的权力，也不延伸到改变由谁来接受判断。

2.2 后一步只在前一步确立的结果上运行

第二种出错是在未知上更新。在例子中，目标留在画面内时的优势可以促成重新出现实验，但把 *A* 推荐到目标离开之后，就是在实验要回答的那个未知上更新。所以后一步只在前一步的实际结果确立其前提时才开始。组合把条件、未知与失败带到下一步（第 3.4 节）；三值检查（three-valued check）的结果有通过、不通过和还说不准三种，还说不准从不当作满足（第 3.3 节）。独立是另一回事：把文献检索与实验分别交给不同执行者，并不会使二者独立，因为检索可能改变假设，也可能表明实验测量的构念并不合适。所以任务分解要找出决定性的未知及相应检查，核对这些结果共同支持什么，继续细分不再改变这一判断时即停止。

细分到不再改变判断就停，这是 Simon 的有限理性 [6] 用在研究者管得过来的证据上。协调理论 [7] 把协调看作管理活动之间的依赖，原则从中取了三条：先就要决定什么达成共识；把工作分成各自能独立推进的部分；事先定好什么情况下要重新分工，问题一变，计划就要重新审视。执行可以委托出去，综合仍是研究者的责任。前一步的字段与后一步的要求对得上，信息就传得过去，但这既不说明它有效，也不说明可以用它；依赖同一项有争议假设得到的一致输出，也算不上相互独立的验证。

2.3 记录比写下它的工具活得久

第三种出错是丢失本该保留的东西。如果 *J* 及其根据只存在于产生它们的那次对话里，论文后来被更正时，就看不出这次更正触及了 *J*。对话窗口关掉、产生记录的插件不再运行之后，早先的记录应当照样读得懂。所以以本地可读、可版本化的来源为准，模型、客户端与插件都可以替换。长期来源、运行时的临时数据、能从来源重建的视图（例如 Dashboard 上的卡片与日历），三者各有分工：视图上显示「已完成」，不等于已被接受；缓存也不会变成另一个事实来源。在原型中，*J* 写在一张卡片上，依据钉在来源版本上；上下文连同回执一起交付（第 4.1 节），适配器不另建事实存储（第 4.2 节）。

记录归研究者所有，这一点取自 local-first 软件 [10]，不过本地为主的存储并不声称实现完整的 local-first 复制模型。按最可能变化的决策划分边界，取自 Parnas [12]。还有，留存不等于检索：记录连同使用条件一起保存，后来的工作才能判断它是否仍然适用；恢复的范围也由当前决定界定。

2.4 纠正只收窄主张，不抹掉历史

第四种出错是用删除来更新。在例子中，更正只触及第二组基准，删掉 *J* 却既会抹掉更正并未触及的第一组根据，也会抹掉第二组上的主张曾被接受过的记录。所以纠正只收窄受影响的主张，不抹掉历史，失败、未知与未验证的状态都如实保留。改了什么、为什么改、哪些部分仍然有用，都留下来：收窄后的判断准入在 *J* 旁边（第 4.1 节），对 *J* 的复核只有经过记录的解决或取消才移出未决集合（第 3.2 节）。例子之外也是如此。已经退役的推荐仍可能提供有价值的反例，所以结论、方法与失败经验都连同适用范围、复核触发与退役办法一起留存。

March 关于探索与利用的分析 [14] 提醒，复用可能挤占探索，所以替代解释与反例都要保留。生成式 AI 与人类学习的研究 [15] 区分了工件的表现与研究者自身理解了什么。照此看，一次成功不确立规则；本文不假定研究者的理解会因此受益，减少或停止辅助的根据也一并保留。

3 研究状态及其变化的模型

原则说了必须保住什么，却还没说怎样才算保住。为了能检查一次维护是否做到，本节在第 1 节的例子上用文字说清保留、更新与挂起，并跟着论文的那次更正走一遍。补充说明 5 给出每条定义的精确表述，并论证第 3.4 节的两条命题；附录 A 把模型中的对象并列在一起，并画出一项要求的生命周期；各定义的实现现状见附录 B.1。

3.1 保留：研究状态与准入

几周之后要接着做那次比较，研究者需要找回什么？不是文件，而是五样东西：在回答的问题，用过的证据及其关系，接受的判断及其范围，仍然悬而未决的条件，以及打算采取的下一步。在例子里，这五样依次是在 A 与 B 之间的选择、论文的两组基准、目标在画面内时的判断 J、重新出现条件下的未知，以及针对这种情形的实验。**研究状态**（research state）就是从记录出发重建的这五样东西，未决条件里还记着冲突，以及判断不适用的情形。这个想法有两类研究可依。分布式认知 [29] 研究借助外部表示展开的推理，研究记录正是这样一种表示；溯源分析 [30] 研究结果如何依赖输入，研究状态里的证据要保存的正是这些依赖。正因为是重建而不是概述，研究状态的来源与状态都可以检查和纠正；它要包含多少内容，取决于问题的要求，而不取决于材料的多少。

来源一变，这个状态跟着怎样变，就是保留、更新与挂起要回答的。一条判断的根据若未被变化触及，它就连同根据与范围一起被**保留**。根据被触及的判断同样留着，只是会有一项**要求**把它标为待复核；要求是一件登记在册的维护工作（第 3.2 节）。研究者准入一个收窄或替换它、并钉在来源新版本上的候选，它才算**更新**。所以在例子里，J 是被更新而不是被删除：它在第一组上的根据未被触及，准入的候选是对它的收窄。准入这个候选本身并不关闭复核，因为要求还带有**检查**，即它必须通过的检验，复核的检查仍须通过（第 3.3 节）。变化触及了哪条根据，模型并不计算，只通过这项复核跟踪每条根据的保留与更新。

挂起与保留、更新不同，它有两个落点，模型将二者分开。证据无法裁定的科学条件留在研究状态的未决条件里，例子中的重新出现条件自始至终就是这样。尚不能裁定的一步维护则处于挂起，并附带一个给研究者的问题，例子中更正之后对 J 的复核就是这样（第 3.3 节）。

谁来改变这个状态？有三方作用于它（图 3）。环境，即图中的 ReSearchOS，从它可以使用的历史中选取上下文，重建研究状态，并连同候选一起呈现；模型提出产出与修改；研究者决定接下来发生什么。无论由哪一方发起，每一次改变都是同一形状：按用途选择输入，检查这次改变可以开始，应用改变，再检查结果。按带类型的图变换 [31] 的说法，我们把这样一次改变称为规则的一次**应用**（application），规则声明它读取什么、必须保持什么、打算产生什么。之所以要这个形状，是因为不同的错误要在不同的步骤上修复（图 4）。论文里一个数读错了，要纠正的是这条观察及依赖它的内容。问的是目标重新出现后的情形，答的却是画面内的排名，就要先澄清用途。只有同类遗漏在多个任务中反复出现，才有理由提议修订规则本身，而这项修订还要在后续任务中检验。

那么，候选怎样变成状态？只有经过**准入**，而且任何一方都无法单独完成。五个条件同时成立，候选才被准入：研究者接受这次修改；授权来自可信宿主或范围内的真实指令，候选自身的文字不能授权；修改确实已经应用；它的检查已经通过；**回执**，即做了什么、查了什么的记录，保存在下一个决策边界读取的历史里（第 3.2 节）。所以，没有运行过的修改，或没有回执的修改，既不算已应用，也不算已检查。研究者的接受定下的是操作层面的决定，判断在科学上是否为真，并不因此而定。

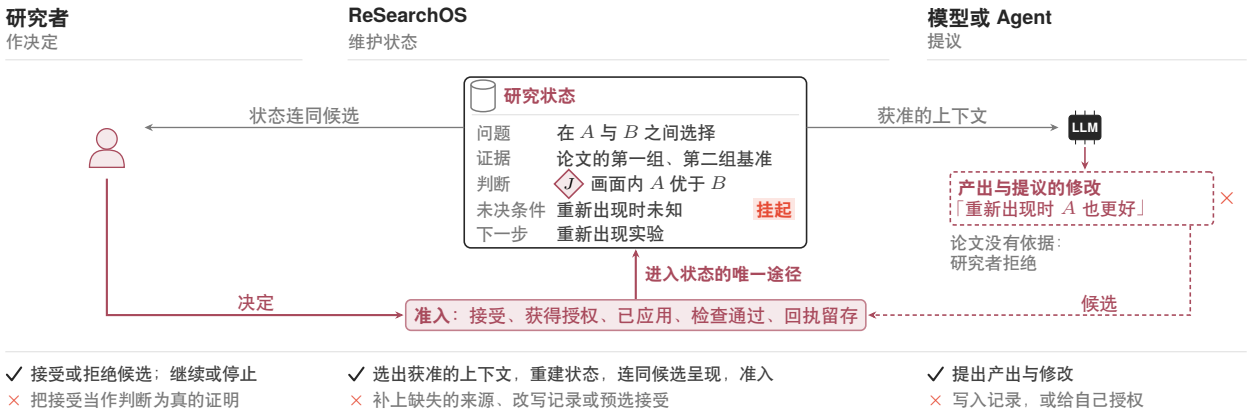


图 3. 只有研究者的准入能把候选变成状态。 ReSearchOS 维护贯穿例子的研究状态，并把它连同候选呈现给研究者。模型或 Agent 拿到获准的上下文，交回的产出与修改只是候选：这里是学生的 AI 助手提出的「目标重新出现后 A 也更好」，论文没有给它依据，研究者拒绝了。进入状态的唯一途径是准入，第一个条件就是研究者的决定（第 3.1 节）。底部两行是各方做什么、不能做什么。

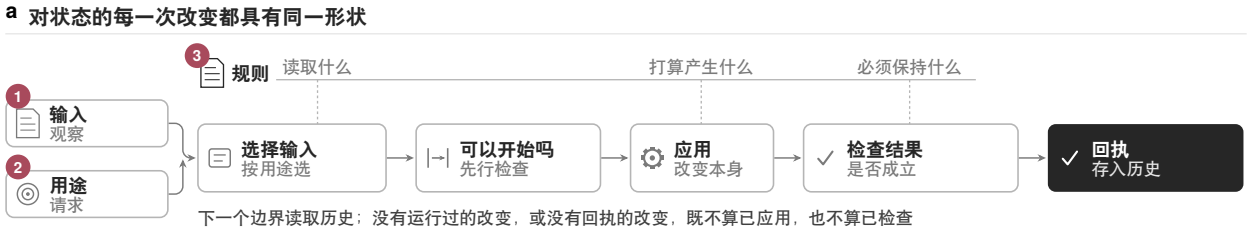


图 4. 对状态的每一次改变都具有同一形状，不同的错误需要不同的修复。 **a**，一次改变按用途选择输入，检查可以开始，应用，再检查结果；规则声明它读取、产生与保持什么，回执存入下一个决策边界读取的历史。**b**，正文的三种错误，编号对应 **a** 中各项修复落在哪一步：读错一个数（1），问此答彼（2），同类遗漏在多个任务中反复出现，这是提议修订规则的唯一理由（3）。

3.2 更新：来源变化触及什么

比较做完几周后，论文被更正，从版本 v_1 变为 v_2。随之而来两个问题：论文现在哪个版本算当前，这次变化又触及哪些工作？系统在**决策边界**上回答这两个问题。决策边界是系统查看已到达的内容、决定下一步的时刻，边界之间工作照常进行。每个边界有一个截点，截点之后到达的记录留到下一个边界再处理。原型里，变化要到下一次 `ros` 命令时才被看到（第 4.1 节）。

当前版本并不就是最新的文件。一个对象的**当前版本候选**，是任务可用的记录中没有已知后继的那些版本；彼此之间没有已知先后的两个版本，于是作为未消解的分支并存，在两者之间选择留给研究者（图 5a）。原型标识版本的方式与此不同（第 4.5 节）。例子里的答案很直接：看到更正之后，论文的当前版本就是 v_2。

这次更正触及哪些工作？以对 J 的复核为例。和每项要求一样，它有稳定的身份，在某一用途下登记，用途带有这项工作的科学条件与审阅依据；它还记下自己依赖的输入，这里是那篇论文，以及复核背后的规则与用途的版本。这些输入中只要有一个自上一边界以来有了新的当前版本，变化就**触及**这项已登记的要求。对 J 的复核依赖这篇论文，所以这次更正触及了它；重新出现实验的根据是未决的重新出现条件，而不是这篇论文，所以不被触及（图 5b）。发现工作也不只靠已登记的要求：初次建立工作或重跑有范围的核对时，扫描还会找出其他需要，包括遗漏的依赖。版本怎样比较、晚到的记录怎样处理、扫描受什么限制，其余规则见补充说明 5。要求一旦被触及，就进入

未决集合，只有经记录的解决或取消才能退出；延期、等待和没有依据的完成报告都不能让它退出，所以没人检查过的工作会延续到下一个边界。

可见，触及标出的是要查看什么，而不是要修改什么。它只沿已记录的依赖走，无法发现从未记录的依赖。假如这次更正只改了一个错字，对 J 的复核照样被触及，J 却没有需要修改之处；反过来，一项新研究即使不改动 J 依赖的任何输入，也可能使 J 不再可靠。文件变化只是需要查看的理由，而不是结论。

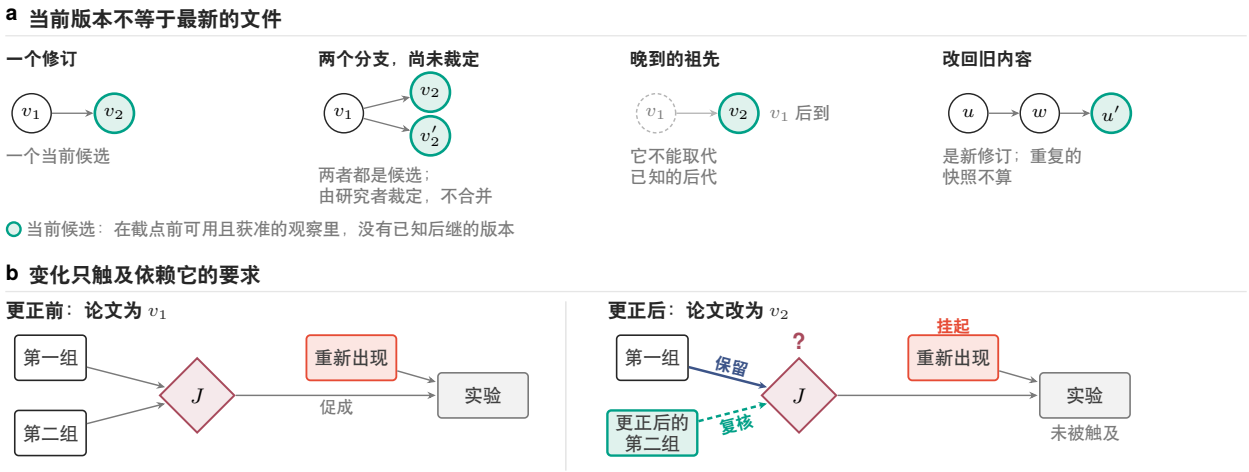


图 5. 变化只触及依赖它或被扫描发现的要求，当前版本也不就是最新的文件。 **a**，一个对象的当前版本候选，是截点前可用记录中没有已知后继的那些版本：图中画出一个修订、尚未裁定的分支、不能取代后代的晚到祖先，以及算作新修订的改回旧内容。**b**，论文更正前后的例子，记号同图 1。J 在第一组上的根据保留，在更正后的第二组上的根据要复核，所以对 J 的复核被触及；根据在未决的重新出现条件上的实验未被触及。

3.3 挂起：三值检查、挂起与等待的工作

论文更正后，对 J 的复核能不能关掉？要看第一组上的结果是否仍然支持 J，而这件事只有研究者能判断。所以系统给每项检查三种结果：通过、不通过、还说不准，分别记作 1、0 和?。只有合格且当前有效的检查确立了，才算通过或不通过；证据缺失、过期或处在未消解的冲突中，就是还说不准，而还说不准从不算作通过。一项要求只有在同一组当前回执上全部检查都通过，才算**解决**。也正因如此，v_1 上的旧回执不能替 v_2 作答，不能用它关闭对 J 的复核。

如果没通过的检查都还说不准，而且每一项都只能由研究者来定呢？这时系统为每一项向研究者提一个选项有限的问题，检查依据的同一版本只问一次，工作就留在这里等回答；有一项检查不通过，就不能挂起。这就是**挂起**。就对 J 的复核而言，回答「仍然支持」，这项检查关闭；回答「不再支持」，复核重新打开；不回答，就一直挂着，不会被算成完成。这是模型对例子的解读，原型则让这项复核保持未决（第 4.1 节）。同一依据版本只问一次，原型的 `ros today` 又把开着的问题集中成一张清单，用意都是减轻审阅负担、防止审阅流于习惯（第 2 节）；这两点都还没有检验过，而且只改一个错字的更正也会开启复核。

工作也可能处于**等待**（waiting），而不是挂起：它等一个会让它回来的触发条件，原因是研究者把它延期或收窄了，或者目标范围之外的东西阻塞了它。把重新出现实验延到预定日期，它就处于等待（第 3.4 节）。图 6 画出各种检查组合允许什么，附录 A 的图 18 画出一项要求的完整生命周期，检查、挂起与等待的其他限制见补充说明 5。对例子要紧的是，挂起与等待的工作留在视野里，却都不解决也不取消要求。只有解决，或取消，才把它移出未决集合；取消是另一个经记录的决定，即不再追求这项需要，它不算修复。即便关闭了，也不是永久的：论文之后再变，可以再次触及同一要求。



图 6. 只有全部通过的检查才能关闭一项要求；否则它挂在问题上、带触发等待、由研究者取消，或仍然未决。 **a**，未决工作在同一组当前回执上每项检查都为 1（通过）时得到解决，或由研究者决定取消。在此之前，只能由研究者裁定的检查各带一个问题时，它可能处于挂起；带触发地延期或收窄、或被范围外的东西阻塞时，它可能处于等待。右栏写每种状态怎样回来。**b**，1（通过）、0（不通过）与?（还说不准）的各种组合及其结果；在例子上，模型把对 J 的复核读作挂起（第 3.3 节），原型让它保持未决（第 4.1 节）。

3.4 组合与有界目标

对 J 的复核挂着，实验也还没做，剩下两个问题：一步何时可以建立在另一步的结果之上，以及工作何时算做完。先在例子上看第一个。目标在画面内时的成功可以促成重新出现实验，却不能拿来在目标离开画面后推荐 A。一般地说，一次应用要建立在另一次应用的结果之上，须有两组条件在当前用途下都已确立：前一次结果所带的条件，以及后一次对输入要求的条件。同一组当前回执通过用途为某个条件声明的每项检查，这个条件才算确立，这与解决是同一个检验。在例子里，比较的结果只对目标在画面内时确立，所以可以促成实验，却不足以在目标重新出现后推荐 A（图 7）。未经检查的条件不为假，但不能放行下一步。



图 7. 后一步只建立在为该用途确立的条件上。 **a**，依据论文比较 A 与 B，得到只在画面内条件下确立的 J。画面内的成功可以促成重新出现实验；在目标离开画面后推荐 A，需要重新出现条件下 A 的优势，而它未知，所以这一步不能开始。**b**，同一组当前回执通过用途为某个条件声明的每项检查，这个条件才算确立。未知不算满足，v_1 上的回执不能替 v_2 作答，格式相容也不是科学前提。

那么，围绕对 J 的复核，工作什么时候算结束？这并不像听上去那样简单：早期原型里，清理宣布完成之后，问题仍会再次出现，前后经历了十轮清理、近两百次提交。所以模型把工作限定为一个**有界目标**（bounded goal），即一个带有冻结范围、冻结规格和明确结果的工作单元（图 8）。范围是目标能读什么：它所在项目拥有的记录，加上经其他项目声明的接口读入的少数输入。规格冻结问题、用途、范围、规则与检查的版本、必须保持的内容、授权、预算，以及带截点的来源版本；它和其他候选一样经过准入。这样，完成与否对照的就是运行中不会变的东西。

在有界目标里，做完和停下是两回事。目标里仍开着的工作，要么挂在问题上，要么在等触发，而且三类不得豁免的工作一项都没开着，目标才算 **Done**。这三类是：范围内运行从未求值的工作、无法重建的前次规格、绕过准入的直接修改仍欠着的确认（见补充说明 5）。它们可以暂时挂起或

等待，但到达 Done 之前必须关闭，所以目标不会靠默认接受自己的结论来收场。目标在三种情况下停下：到了 Done，预算的某一部分用完，或研究者决定停止；停法不同，记法也不同。Done 且没有未决工作，记为完成（complete）。Done 但还有挂起或等待的工作，记为 complete_with_held：逐项列出这些仍开着的工作及其问题和触发条件，并报告目标已登记的要求里已解决的比例；比例为零记为延期（deferred），不算完成。否则，因预算用完而停，记为 budget_stopped；由研究者叫停，记为 human_stopped；两者都不报告完成。

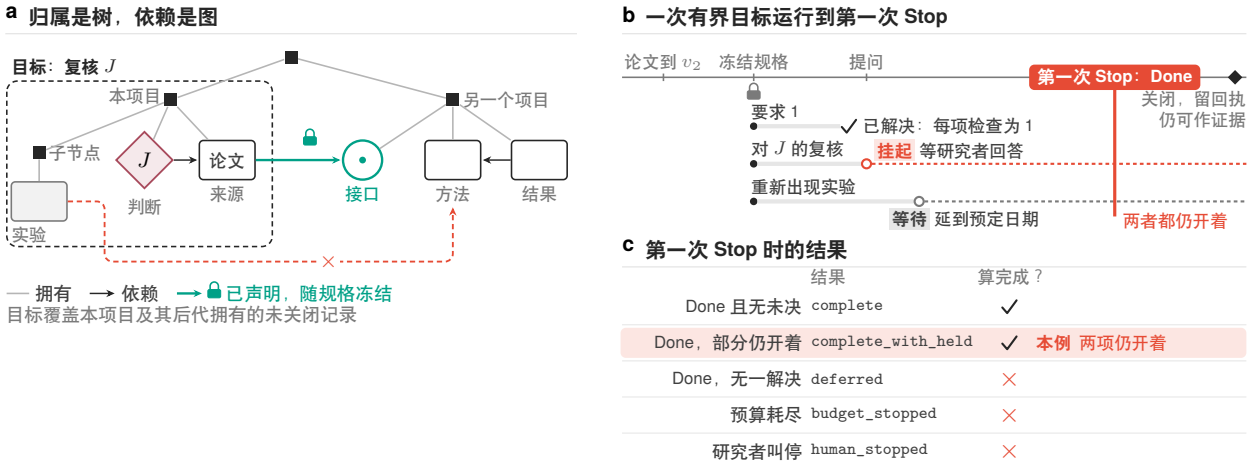


图 8. 有界目标冻结它能读什么、要答什么，挂起与等待的工作报告为仍然开着。 **a**，归属（灰线）是一棵树，按记录所在位置给每条记录一个归属；依赖（箭头）可以连接任意记录。复核 *J* 的目标覆盖它所在项目节点及其下各节点的未关闭记录，只经另一个项目声明的接口读取那个项目，接口随规格一起冻结；绕过接口的读取落在冻结范围之外（补充说明 5）。**b**，按正文设想的那次延期运行。规格记下论文的 v_2，途中要求 1 得到解决。按模型的解读，对 *J* 的复核挂起，实验处于等待，所以目标 Done，两者都仍开着；有回执的关闭之后，目标仍作为证据可用。**c**，第一次 Stop 时读出的结果，为 complete_with_held。

图 8b–c 在一个设想下画出例子：研究者还在终端把重新出现实验延到预定日期。这时实验处于等待，对 *J* 的复核挂起，第三项要求（图 8b 中的要求 1）的检查全部通过而得到解决。三项要求中解决了一项，在研究者回答之前目标就已 Done，结果为 complete_with_held，两者都仍然开着。第 4.1 节的贯穿会话没有做这次延期，所以那里的实验仍只是未决，目标也不算 Done。

这些定义最终能保证什么？两条命题回答这个问题，它们的假设与论证见补充说明 5。第一条说，冻结范围互不重叠的两个目标没有共同的要求，所以一个目标的完成不依赖另一个：比如一个无关项目上的目标，既不增加也不减少复核 *J* 的那个目标的要求。第二条说，目标经过有限步，要么到达 Done，要么耗尽预算；除了假设，它还需要一个调度上的条件，因为重开一项要求、或回答「不满足」，都会增加开着的工作。两条命题都只在这些条件下成立，而决策边界的预算有限，保证的是目标会停下，而不是会完成。两条命题都不证明完成可达，那还需要每项未决要求都能在范围与预算内裁定，并且没有新的来源变化。

4 原型：在命令行上保留、更新、挂起

这个模型有多少已经落到终端上？保留、更新、挂起这三个动作，连同研究者在其间所作的决定，各有一部分已经成了命令行上的命令。本节描述附录 B.3 所列两个提交上的本地原型，它们的测试情况也概述在那里。

4.1 贯穿案例：论文更正后的一次会话

会话沿用第 1 节的例子，方法 A、B 均为虚构，不报告任何真实方法的效果。图 9 画出这次会话，共六步，每一步是 `ros.py`（运行一次会话的程序）的一条命令。会话之前，研究者已把 J「目标在画面内时，A 优于 B」绑定到论文的 v_1 版本，根据是两组基准；重新出现条件下的未知留在未决条件中，重新出现实验也已计划好。随后论文更正为 v_2：第二组的数字改变，第一组不变。

有一处与模型的差别贯穿整个会话，所以先说。按模型，对 J 的复核挂在给研究者的问题上（第 3.3 节与图 6）；在这次会话里，原型却让它保持未决。原因在于原型能查什么。这项复核上，原型自己能裁定的检查只有一项：依据是否钉在当前版本上。更正之后，这项检查不通过（返回 0）。不通过不是「还说不准」，没有可挂起的，复核就只是开着；第一组的结果是否仍支持 J，留给研究者判断（差距 34，语义检查；差距编号见第 4.5 节与补充说明 6）。原型只在遇到自己裁定不了、必须交给研究者的检查时才挂起：`ros today` 只问一次，研究者用 `ros answer` 回答之前，这项要求一直挂着。不过随附的规则里，没有哪一条声明了这种以研究者判断为依据的检查，所以挂起路径在会话内核的回归测试里走到过；在测试之外，只有要求的主体或依据无法解析时才会走到（附录 B.1）。

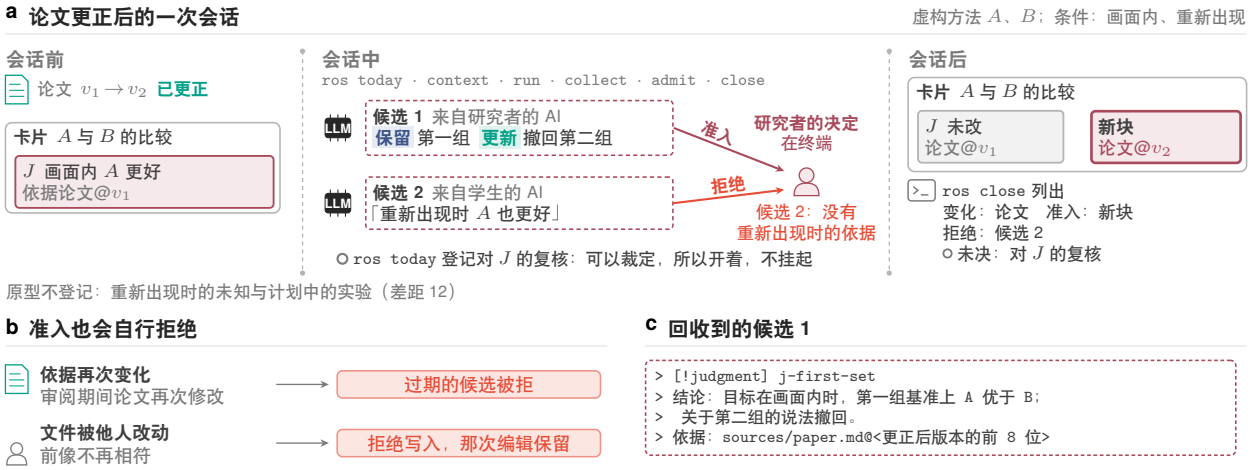


图 9. 一次会话里，收窄的判断凭研究者的决定准入在 J 旁边，无依据的提议被拒绝，复核仍然开着。论文更正后的贯穿会话（虚构方法 A、B；条件为画面内与重新出现）。**a**，会话前，论文已更正到 v_2，卡片上的 J 仍以论文的 v_1 为依据。会话中（`ros today`、`context`、`run`、`collect`、`admit`、`close`），`ros today` 把对 J 的复核登记为未决工作；这项检查可以裁定，所以它开着，不是挂起。会话回收两个候选：一个来自研究者自己的助手、经可复制任务包交回，保留第一组、撤回第二组，凭研究者在终端的决定准入；另一个来自学生的 AI 助手，声称目标重新出现后 A 也更好，因论文对重新出现没有依据而被拒绝。会话后，J 未改、仍是 v_1，旁边多了以论文 v_2 为依据的新块；`ros close` 列出变化、准入、拒绝，以及对 J 的复核仍开着。**b**，准入自行拒绝的两种情形。**c**，回收到的第一个候选。重新出现条件下的未知与计划中的实验是模型层状态，原型不登记（差距 12）。

更正落到记录上的方式是：用户，或按用户指示行事的 Agent，替换记录这篇论文的本地文件，这里是 `sources/paper.md`；下一条 `ros` 命令便看到它的内容摘要变了。会话由 `ros today` 开始：它发现版本变化，把对 J 的复核登记为未决工作，但此时并不提问。更正可能只改了一个错字，判断依旧成立，保留还是更新尚未确定。

接着，`ros context` 交付获准使用的来源与当前状态，并附回执；`ros run judgment --target <card> --executor copy_packet` 为承载 J 的卡片写出一个任务包，不调用任何模型服务。任务包是装有任务与已交付上下文的 Markdown 文件，由用户粘贴到任一助手中。随后，`ros collect <run> --file draft.md` 把助手的草稿读回，作为候选。草稿中只读取一到三个判断块，其余内容一律忽略：

```
> [!judgment] j-first-set
> 结论：目标在画面内时，第一组基准上 A 优于 B；关于第二组的说法撤回。
> 依据：sources/paper.md@< 更正后版本的前 8 位>
```

这里的依据把论文钉在 v_2 上。依据按来源文件记录，即文件路径加内容摘要，所以原型看不到论文内部第一组与第二组的划分；更正触及哪一部分，由候选写明（差距 11）。不带版本的依据，按回收时的当前版本钉住；钉在已过期版本上的草稿会被拒绝。该候选不动重新出现条件与实验，因为二者另有独立根据：撤回 *A* 的一部分优势，既不推出 *B* 更好，也不推出实验应当取消。

接下来由研究者决定，`ros admit <candidate>` 执行准入。它要等研究者审阅之后，才把收窄后的判断作为新的判断块写在 *J* 旁边；确认是在终端作出的，所以这次准入记为研究者的决定。在原型中，更新指的就是这一步：*J* 仍留在卡片上，依据仍钉在 v_1，保留与更新都落在旁边那个新块上，原型不修改 *J* 本身。

同一次会话中，又从学生的 AI 助手那里回收了第二个候选「目标重新出现后 *A* 也更好」，依据同样钉在 v_2。论文没有报告目标重新出现后的任何结果，研究者便在终端拒绝了它。拒绝记为研究者的决定，不写入任何内容，状态也不变。会话以 `ros close` 结束：它比较会话起止时的来源，列出改动了什么、准入了什么、拒绝了什么、哪些候选仍未决定，以及哪些事项仍未关闭。

准入也会自行拒绝，情形有两种，都不是科学判断。候选审阅期间若论文再次修改，准入以依据过期为由拒绝该候选，要求在当前材料上重新提议。若承载 *J* 的文件在写入计划形成之后、实际写入之前被他人编辑，文件的前像（形成写入计划时所依据的内容）就不再相符，系统于是拒绝写入，不覆盖那次编辑。审阅期间他人所作的编辑则会保留，因为写入计划要到决定作出之后，才按文件当时的内容形成。

公开 starter 在默认的只复制模式下，于自己的虚构夹具（经 `--root` 指定）上运行同样的命令，因此它的运行不能作为这次私有会话的证据。如表 3 所示，starter 可以在本机写入运行态、任务包与候选，但只产出可复制的候选或拟议差异，从不准入候选，也从不改动研究者的来源：

表 3. 贯穿会话与默认公开候选路径

命令	本次私有终端会话	默认公开 starter
`ros today`	观察来源变化，把判断标为待复核	查看虚构来源，维护 Git 忽略的工作状态
`ros context`	交付获准来源与重建状态，留存回执	交付获准虚构上下文，留存回执
`ros run judgment --target <card> --executor copy_packet`	准备任务包，不调用模型服务	准备可复制任务包，仍拒绝 provider 派发
`ros collect`	将草稿回收为绑定来源版本的候选	回收候选，不准入长期来源
`ros admit`	经研究者终端决定准入；过期依据或冲突前像被拒绝	通过复制回退显示拟议差异，不写入来源
`ros close`	列出变化、已准入、已拒绝、待决候选和仍开着的工作	记录本机会话与残留工作，不向长期来源准入

4.2 职责划分与稳定内核

这些命令背后，谁拥有什么？第 2.3 节要求模型、客户端和插件可以替换，而研究者的来源始终是权威；图 10 画出为此设定的目标划分，共分五行：客户端、方法、稳定内核、适配器与记录。长期来源归研究者所有；稳定内核从适配器提供的快照计算变化与未决要求，从不直接读取来源。适配器经既有服务处理文件、执行与回执，不另建事实存储；客户端负责组织呈现；模型输出可以提议，但不能授予写权。在原型中，运行第 4.1 节命令的会话内核是 `ros.py`，它背后的治理合同（规定什么可以成为长期状态）与协调合同（规定现在要处理什么）同属一个包。准入本身却仍由 `ros.py` 而非该包执行，所以图中单一的稳定内核仍是目标，而非现有代码（差距 3）。补充说明 2 收录本节背后的命令、记录表与逐步记录。

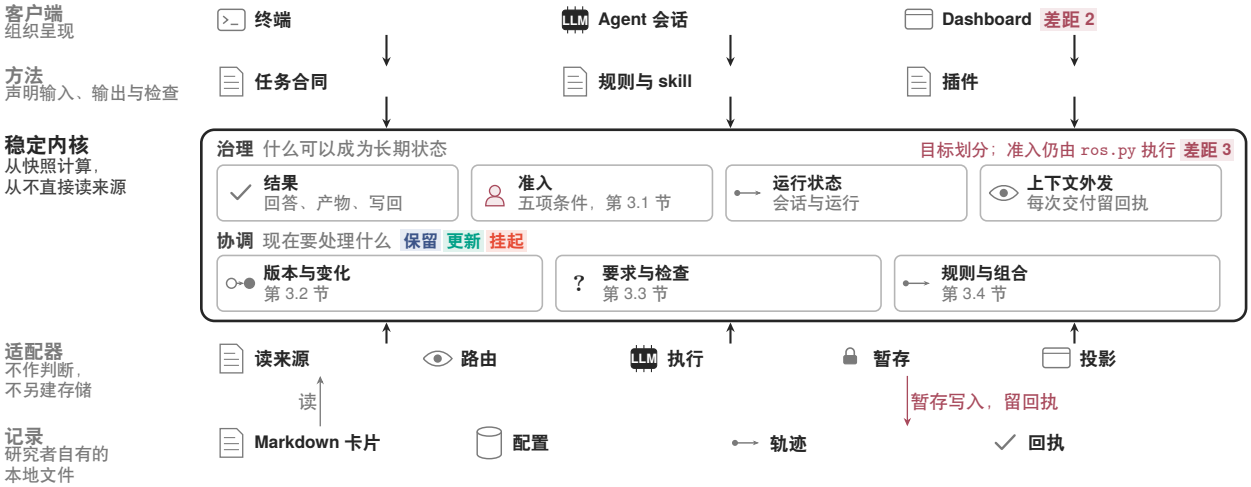


图 10. 所有依赖指向一个从不直接读来源的稳定内核；本图画的是职责的目标划分。客户端与方法自上、适配器自下依赖内核。内核从适配器提供的快照计算，规定什么可以成为长期状态（治理）和现在要处理什么（协调）。适配器读记录，只以带回执的暂存编辑写入；模型输出只作为候选进入。Agent 会话包括 Codex、Claude Code 等；配置是 `config/ros.json`。酒红色标签标出原型与之不同的地方：Dashboard（差距 2），以及仍自行执行准入的 `ros.py`（差距 3）。

内核背后有四样东西：记录图（即记录及其之间的链接）、规则、未决要求和执行轨迹。它们是逻辑上的结构，不是四套数据库：长期来源仍是 Markdown 文件和明确作出的决定，要求处于什么状态由已有的轨迹推导出来，缓存只是让恢复更快。轨迹能确立决定了什么、尝试了什么、观察到什么、检查了什么，但确立不了从未记录的依赖，对长期来源也没有权威。原型还有其他内部用语，补充说明 2 逐一列出，并写明各自承担的合同；只有带着不同的合同或生命周期，才值得另立一类。这些边界是设计上的承诺，实现是否守住，要到实现里去检查。

4.3 会话协议与准入

第 4.1 节的收窄判断为什么不能像一条事实那样直接写入？因为事实与判断经由不同的路径进入长期状态（图 11）。事实可以在既有任务授权内更新；判断则要经过候选、决定、实际修改、检查与回执，每一步分别记录。候选经 `ros collect` 或 `ros propose` 记录时，它所依赖的每项依据都按完整的来源版本冻结。审阅期间若依据发生变化，准入拒绝过期的候选并要求重新提议；已存在的判断只有经显式的重新确认，才能重新绑定依据。

写回还必须落在形成计划时的那份内容上。准入、重新确认、关闭 episode 卡片（`ros propose episode-close`）和规则修订这四条路径，写入都分两步：先由暂存服务保管计划中的写入，不碰来源；再由升格把它写进来源。episode 卡片是一张 Markdown 卡片，它记录的状态标明一段工作开着还是已关闭。形成写入计划时，这些路径把原文件摘要交给暂存服务；摘要是形成计划时那份内容的指纹，也就是第 4.1 节所说的前像。升格时前像若不符，就拒绝写入，保留其间的编辑，也

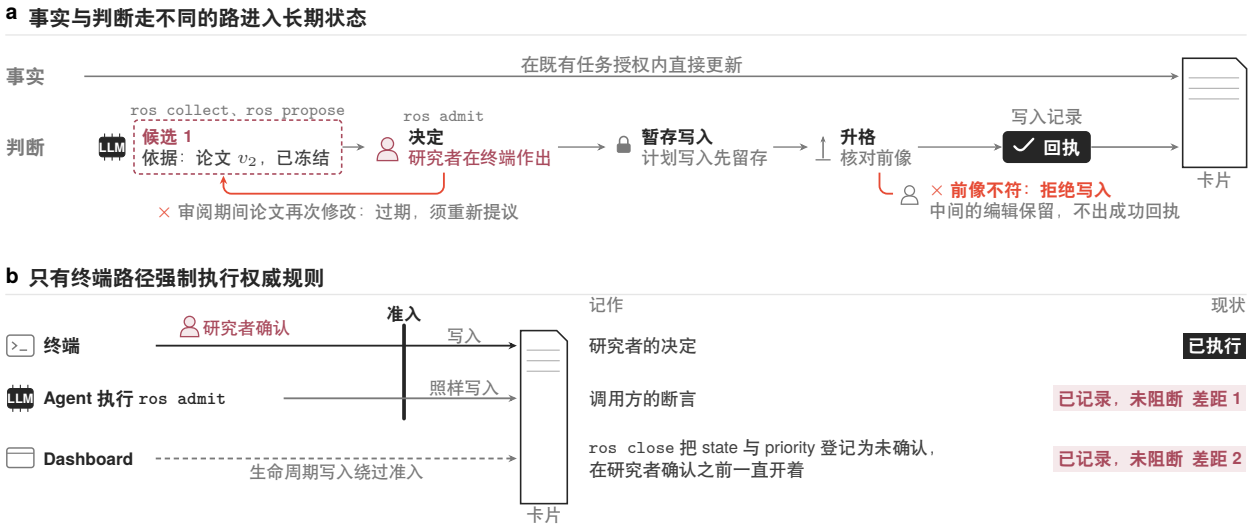


图 11. 事实与判断走不同的路进入长期状态；能写同一张卡片的三个入口里，只有终端记为研究者的决定。 **a**，事实在既有任务授权内直接更新。判断（这里是第 4.1 节的候选 1，依据是 v_2 版论文，由 `ros collect` 或 `ros propose` 冻结）要经过研究者在终端的决定（`ros admit`）、暂存写入、升格与回执。审阅期间依据变了，候选作为过期被拒绝；文件的前像不再相符，写入被拒绝，中间的编辑保留，不出成功回执。**b**，三个入口写的是同一张卡片。在终端的准入记为研究者的决定；Agent 执行 `ros admit` 记为调用方的断言，照样写入（差距 1）；Dashboard 的生命周期写入绕过准入，`ros close` 把其中的 state 与 priority 登记为未确认的修改，在研究者确认之前一直开着（差距 2）。

不发出成功回执，这就是第 4.1 节说的那种拒绝。这些检查保护内置的写回路径，但不验证任意规则须保持的部分（差距 9）。

那么，一次准入记下的是谁的决定？这要看它从哪个入口进来。在终端确认的判断准入记为研究者的决定；由 Agent 执行的准入记为调用方的断言，照样写入，所以对能使用 shell 的 Agent 而言，这是归属与审计记录，而不是阻断。终端靠交互式提示来识别，分配了伪终端的进程也能应答这个提示，其应答随后同样记为研究者的决定：记录证明的是通道，而不是人（差距 1）。Dashboard 是原型的图形客户端，它的生命周期写入绕过准入。若写入发生在会话期间，`ros close` 把其中承载判断的字段（state 与 priority）登记为未确认的修改，研究者确认之前不会关闭（差距 2）。图 11b 画出这三个入口；第 4.5 节把这两处列为最要紧的差距。

关闭一项要求则更严格。`ros decide … cancel/confirm` 与 `ros answer … cancel/satisfied` 只在终端接受，Agent 的转述只记为调用方的断言，要求仍不关闭。对未经准入就写下的块作事后确认，也遵循同一规则：由 Agent 执行的 `ros admit --confirm` 只记为调用方的断言，确认要求仍不关闭。收尾时也不重复提问：有些承载判断的字段在同一会话内已由完整的决定与成功的回执覆盖，就不再交给研究者确认第二次；之后的直接修改或不完整的变化链仍需确认。因此，一次准入既不要求研究者把刚作出的决定再作一遍，也不授权之后的任意修改。

4.4 客户端、方法与公开边界

终端之外的客户端又处在什么位置？命令行是明确使用内核结果的客户端。Dashboard 上的卡片、选择与日历是投影，也就是记录的视图，所以既不建立科学关系，也不算接受。方法的处境类似：可以声明输入、输出与检查，但声明并不给它接受的权力。已有种类的规则只要一个 JSON 文件；新种类的可执行检查却要改内核、过架构审查，在那之前，它留下的未评估工作会挡住完成（差距 12）。各客户端是否守住这些边界，要逐个检查：不带界面的回归测试，既不说明图形交互能通过验收，也不说明外部 Agent 受到控制，这些 Agent 的宿主保留各自的权限。私有部署与公开部署的科学含义相同，权限不同；任务包复制出去，不带走任何权限。

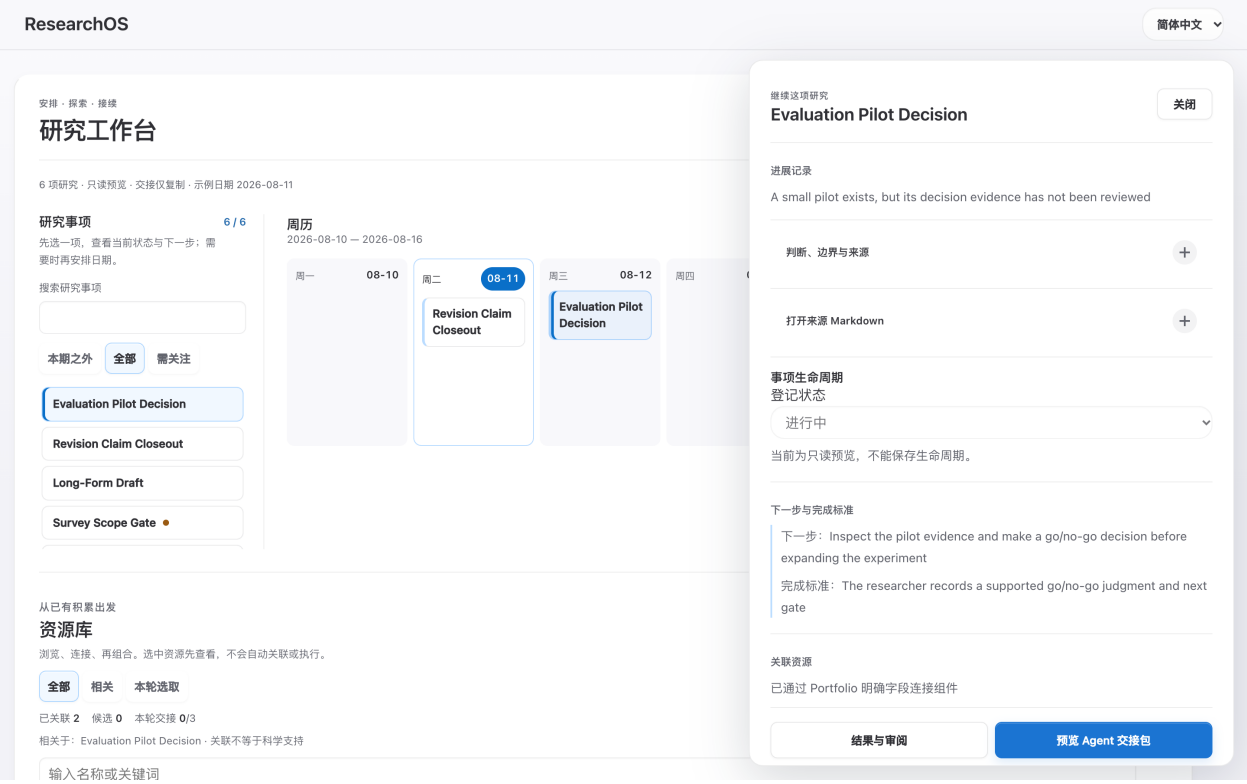


图 12. 公开 starter 为研究者重建一个事项的状态，但不能保存它。公开 starter 的研究工作台，打开的是虚构事项 Evaluation Pilot Decision。截图于 2026 年 10 月 29 日取自发布版公开 starter 的全新克隆（提交 f541cb6），未经修饰。右栏重建这一事项的状态：进展记录，判断、边界与来源，以及下一步与完成标准。公开 starter 不保存事项状态：生命周期不能保存，Agent 交接包只能预览和复制。

Dashboard 首页的设计起初考虑以问题、根据、未决条件与下一步作为主入口。作者日常更习惯日历和资源卡片，所以首页保留了这一熟悉的入口，研究状态则放在选中事项的详情中。把资源选入交接只是临时组装上下文，而记录它支持某个结论则是需经准入的断言，所以二者不宜共用同一个拖拽动作。其他研究者会觉得哪种安排更好，目前还不知道，作者的偏好与实现检查都说明不了这一点。

试用。公开 starter 是只复制的版本。它出厂的 `config/ros.json` 设定 `runtime_scope: public_starter`，下文所述行为在这一配置下成立。在它的虚构夹具 `examples/scheduler-demo` 上，第一次会话按第 4.1 节的步骤进行，命令取表 3 的公开一栏。starter 不向任何模型 provider 派发，也不向长期来源（即算作权威的记录）准入任何内容，所以用户能体验恢复、上下文交付与候选回收，体验不到准入。图 12 是公开 starter 的研究工作台，图 13 是它的 Agent 交接，公开 starter 只能预览和复制交接包。两张截图展示的是 starter 怎样呈现一个事项，不表示任务已完成或判断已被接受，也不说明界面的可用性。

私有版增加了准入与 Dashboard 写回。其会话内核还可以把一次有界运行交给本地 runner，即 Codex CLI 或 Claude Code CLI，二者各自须在 `providers` 下声明为可派发。两者都从空的临时工作目录启动：Codex 使用只含登录凭据的临时配置目录，在只读沙箱中运行，网络关闭；Claude Code 在受限模式下运行，因此不读取用户设置，也不启用工具。公开 starter 在 CI 中以 CPython 3.10 与 3.13 测试，私有版以 CPython 3.11 测试（附录 B.3）。

私有版运行在作者本人的记录上，不对外分发。想在自己的 Markdown 笔记上使用的用户，可以用 personal-vault 模式启动公开 starter。这一模式从这些笔记生成本机的 Dashboard 视图，经用户确认



图 13. 交接只是用户复制的文本；不运行任务，也不改记录。公开 starter 的 Agent 交接预览，打开的是图 12 中的同一事项。截图于 2026 年 10 月 29 日取自发布版公开 starter 的全新克隆（提交 f541cb6），从对话框顶部截到交接包开头，未作其他修饰。说明框上方是本轮目标与选定的材料。预览说明它只准备交接文本：不运行任务，也不修改研究记录。展开后可见用户复制给所选助手的交接包的开头，即摘要值与 copy_only 路由；交接包随后依次是目标、会话来源与上下文路线。

后把 Dashboard 的修改写入 `personal_vault/`；自行配置后，还可以启动本机安装的 Codex CLI。`ros` 本身不随这一模式改变，`ros admit` 仍只显示拟议差异。`ros` 不拦截普通的 Agent 会话，这些会话照常进行；只有经 `ros collect` 回收或经 `ros propose` 登记的判断块，才进入候选路径。安装耗时与每次会话的投入尚未测量。

4.5 原型还不会做什么

原型离第 3 节还有多远？第 3 节是规格，其精确表述见补充说明 5；当前代码提供其中若干可运行路径，实现覆盖按义务逐项登记，不由测试数量推断。表 4 按主题列出最要紧的差距，补充说明 6 给出全部 39 项及其完整说明。这些差距限定的是实现能主张什么。第 3 节的模型、它的定义及其条件论证并不因此改变；第 4.2 节的架构规定谁对什么负责，并不表示每种组合或客户端都已存在。

对原型使用者而言，最要紧的差距有三项。其一，只有终端路径执行权威规则：Dashboard 写入与 Agent 执行的准入如第 4.3 节所述被记录下来，但不被阻断（差距 1、2）。其二，来源变化要到下一次 `ros` 命令时才被看到，因为目前没有文件监听，变化也不会跨多跳传播（差距 34）。其三在等待上：研究者在终端用 `ros decide … defer` 确认的延期确会产生等待（附录 B.2），但还没有机制会因范围外依赖而产生等待（差距 17）。

除第 4.1 节所说的挂起路径外，原型还在另外几处偏离模型。一次更正触及判断的哪些部分，由候选写明、由研究者决定，原型并不计算（第 4.1 节，差距 11）；原型也不能替换或撤回 J，J 由研究者手动修改或删除。按模型，重新出现条件下的未知仍留在未决条件中，实验仍是一项未决要求；但原型没有规则登记研究者陈述的条件或计划的实验，所以 `ros close` 不列出它们（差距 12）。只有研究者在终端用 `ros decide … defer` 把实验延期，实验才处于等待，贯穿会话没有这样做。此外，原型只按内容摘要标识版本，改回原先的内容算回到原先那个版本，而不是一个新修订（附录 B.1）。它也没有实现完整的条件组合（差距 10），会话只近似有界目标（差距 15），跨客户端集成仍未验证。

两类证据要分开。内核与写回的回归测试在临时夹具上演练来源变化、提议冻结、准入、并发写入拒绝与收尾记录，测试通过确立的是这些输入下的执行性质。至于画面内条件与重新出现条件下的判断是否正确、实验是否值得做、研究者是否理解得更多，则要由实际任务和独立证据来回答。

表 4. 按主题分组的已披露实现差距

主题	差距编号（对应补充说明 6）	对原型使用者意味着什么
准入与写回	1、2、9	Agent 执行的准入与 Dashboard 写入只被记录、不被阻断；内置检查也不验证任意规则须保持的部分。
有界目标与完成	15、17、18	会话只近似有界目标：规格冻结不完整，范围外依赖的等待尚未产生，过期依据上的挂起问题仍可能让会话报告 complete_with_held。
组合与维护	3、10、11、12、34	单一的稳定内核仍是目标，因为准入仍由 `ros.py` 自行执行。通用组合检查、更正时对有独立支持部分的保护、陈述条件与计划实验的登记、文件监听与多跳传播都只停留在规格。
验收与收益	13、14	界面的真人验收、真实模型的隔离、长期恢复与科研收益都尚未验证。

5 相近系统、最强反证与结论

即便补齐了这些差距，还剩一个问题：研究者已在使用的工具，是否已经做到 ReSearchOS 要做的事？如果没有，一个配上好交接的强模型是否就够了？前一半可以把 ReSearchOS 放到相近系统之间来对照；后一半只有第 5.2 节所说的那项比较才能回答。

5.1 相近系统

最接近的替代品不是科研 Agent，而是研究者已经在使用的工具。比较它们最直接的办法，是让贯穿例子中的论文在每一类工具里都更正一次。表 5 逐类记下保留什么、来源变化时更新什么、能挂起什么；每一行描述的是该类中点名的或典型的系统所报告的做法，并不涵盖该类的每个系统。图 14 画出其中六类在这次更正下的情形，下文只补充表中一行说不清的内容。在笔记工具里什么都不会发生：记着 J 的笔记原样不动，直到研究者亲手或让助手去改。版本控制保留论文记录的每个版本，用乐观并发控制 [32] 拒绝过期写入，还像版本向量 [33] 那样检测并发分支；乐观并发控制也正是原型前像检查所用的机制。但版本控制不区分事实与判断，遇到冲突的处理方式是合并。ReSearchOS 则把冲突分支保留为当前版本候选（补充说明 5），把选择留给研究者。

有几类系统会自己处理这次更正。真值维护系统 [34] 记下每条信念为什么成立，并沿这些理由修订信念：撤回某个假设的理由一旦失效，这个假设就恢复；还有别的有效理由支撑的信念，则继续成立。把两组基准画作两条独立的理由，经第二组的理由失效，J 便凭第一组继续成立（图 14c）。

基于逻辑的真值维护系统 [35] 把信念标为真、假或未知；基于假设的真值维护系统 [36] 同时保留所有相容的语境，不必撤回。第 3.3 节的检查用的也是这三个值；第 3.2 节的被触及集更简单，只沿已记录的依赖走一步，标出被触及的要求。两者的区别在于由谁给值。真值维护系统由系统自己重新标注，「未知」是指传播既没推出真，也没推出假；ReSearchOS 里，系统裁定不了的检查，在冻结范围内由研究者给值，所以一次变化可以让判断挂起、留给研究者。AGM 传统的信念修正 [37] 用受一组公设约束的函数定义理论怎样改变；本文的保留、更新与挂起说的则是研究者的记录发生了什么，不是修正算子。

构建系统 [38] 同样自己处理变化。它决定一个输入变化会使什么失效，支持提前截止的系统在重算值未变时即停止，这就是对工件的更新动作；但被暂停的构建任务等待的是依赖就绪，而不是人的判断。Agent 记忆保留并检索。Graphiti [39] 不删除被反驳的事实，而是使其失效；GitHub Copilot Memory [40] 在使用一条记忆之前，先对照当前分支核对它所引用的代码位置；PACMI [41] 把依赖于已变化证据的记忆标为待核实。三者都由系统设定新的状态，不询问任何人。GovMem [42] 把有疑问的候选记忆交给人审阅，但只在写入之前，也没有描述记忆写入后的复核。AI 科学家系统在自身的权威下保留、更新并产出工件：Agent Laboratory [43] 在每个阶段接受研究者的反馈；Kosmos [44] 在最长十二小时的一次运行内，为报告中的每条陈述标出代码或文献出处。它们都不记录经过准入、带有范围、来源变化后会被重新打开的判断。

科研知识图谱与实验跟踪器更新的是记录，而不是建立在记录上的判断的依据。科研知识图谱 [45] 会把论文及其比较做成一等对象，并为存储的数据保留版本与溯源；本文的判断协议在此之上加入范围、检查与权威，用来决定建立在这项比较上的判断是否仍然成立。实验跟踪器与溯源存储保留运行与谱系；它们回答执行过什么，而不回答判断是否仍然成立。

另一些工具把变化交回给人，这种做法并不始于本报告。IBM Engineering Lifecycle Management 这类需求管理工具在链接两端任一制品的内容发生变化时，把追踪链接标为可疑，再由团队成员标为有效或无效 [46]。在 GitHub 上，受保护分支可以设置为：新提交改变了已批准的 diff 之后，这次批准即作废，改动须有人重新批准后才能合并 [47]。活体系统综述以类似的方式把研究结论交回作者，通常是在发现新证据时：作者判断新研究是否可能对综述的结论产生重要影响，在整合新证据期间，读者会被告知更新尚待完成 [48]。2026 年 6 月，Cochrane 宣布正在把已撤稿的研究与纳入它们的综述对应起来，还宣布将联系作者、在评估影响期间为受影响的综述加编辑注，结论如何处理则交由编辑决定 [49]。更接近单条主张的是一篇论文的定量主张台账 [50]：它为每条主张所依据的数据记下指纹。数据一旦变化，即使主张仍然通过也标为过期，并阻断依赖于某条未通过主张的其他主张。

这些工具挂起的是一条链接、一次待合并的改动、一整篇综述或一整条主张。没有一个按范围与依据版本保存已准入的研究判断、只经研究者准入的候选更新变化触及的部分、并把带问题的检查挂起给研究者。反过来看，一项新研究即使不改动判断所依赖的任何输入，也可能使判断不再可靠（第 3.2 节）。活体综述正是为这种情形设计的，ReSearchOS 则察觉不到。表 5 中 ReSearchOS 这一行写的是设计机制，其中多少已经可以运行，见第 4.5 节。



图 14. 论文更正时，各类相近系统要么自己处理这次变化，要么留给研究者去改。 **a**–**f**，同一次更正（各格左侧）落在六类相近系统上，按图 1 的保留、更新、挂起配色画出。**a**，笔记工具让链接的笔记原样不动，等研究者手改或经由助手去改。**b**，版本控制保留每个版本，合并并发分支，拒绝过期写入。**c**，真值维护系统自行重新标注：第一组、第二组画作两条独立的理由，经第二组的理由失效，*J* 凭第一组继续成立。**d**，构建系统重建结果，人在环外。**e**，Agent 记忆由 Agent 决定覆盖旧片段或判定其失效。**f**，AI 科学家系统在自己的权威下重跑，研究者可对某一阶段给出反馈。每格底行写明由谁处理这次变化；六类之中没有一类把它挂起、作为问题留给研究者，表 5 的最后一列也这样记录。各类保留什么、更新什么、能挂起什么，由表 5 用文字给出；图画的是这次更正在各类系统里怎样走。

表 5. 按三个动作比较相近系统

系统类别	保留什么	来源变化时更新什么	保持开放的是什么
个人知识库与笔记工具	笔记与链接	来源变化时不更新；研究者手改，或让助手改	无；未决问题只是一条笔记
带乐观并发的版本控制	每个版本；拒绝过期写入	合并，或报告冲突	分支，但不说分支意味着什么
真值维护系统（JTMS、LTMS、ATMS）	结论及其理由	自行沿理由重新标注；仍有别的有效理由的结论继续成立	未知标签（LTMS）或同时保留多个相容语境（ATMS），由系统处理，不交给研究者
撤稿处理与主张台账	综述及其纳入的研究，或带证据指纹的主张	提示被撤稿的研究，或在数据变化时把主张标为过期	一整篇综述（Cochrane 宣布将为其加编辑注），或作者更新之前的一整条主张；不按范围，也没有待准入的候选
构建系统与科研工作流	键、值及其输入	重建被变化失效的部分	无；人在环外
科研知识图谱与实验跟踪器	比较记录、运行与谱系	记录本身，不是判断的依据	无
Agent 记忆（RAG、A-MEM、Mem0、MemOS、Graphiti、PACMI）	检索与压缩后的上下文	覆盖、重排，或由 Agent 判定失效	至多是系统设定的待核实标记；不交给研究者
AI 科学家系统（AI Scientist、Agent Laboratory、Co-Scientist、Kosmos、Denario）	假设、实验与工件	在自己的权威下重跑；研究者可对某一阶段给出反馈	没有由研究者决定的东西
ReSearchOS	带范围与依据版本的判断	只更新被触及的部分，经研究者准入的候选	未决的科学条件，以及带一个给研究者的问题的检查

研究者权威本身也有先例，对每一个都可以问：论文更正之后，那次批准怎么样了？OmniScientist 的 OSP 协议 [51] 把人对某个工件的批准连同该工件的版本哈希，记入中心 Hub 上的溯源链。被批准的是 Agent 提交审阅的工件，而不是带有范围、来源变化后会被重新打开的判断；在贯穿例子中，若批准的只是「*A* 优于 *B*」这一排名本身并把它传给下一项任务，就会掩盖未检验的重新出现条件。两个新近的系统在研究者权威上更接近。Magnolia [52] 提出的跨项目规则，要等研究者逐条确认之后才会应用；ResearchLoop [53] 只在门槛（gate）所批准的范围内准入一条主张，门槛由研究者或预先指定的协议文件编写。但两篇论文都没有描述在来源后来发生变化时把已准入的结果交回研究者。在 Magnolia 中，工具版本被取代后，绑定在旧版本上的规则会一直保留，直到研究者修订或停用它；在 ResearchLoop 中，复核是运行时对门槛的确定性评估，证据链缺失或不匹配的主张会被降级或阻断。图 15 按来源变化之后保持开放的是什么、由谁处理，摆放表 5 中的多数类别和本节点名的几个系统。



图 15. 相近系统与 ReSearchOS 的不同，在于来源变化之后保持开放的是什么，以及由谁处理。表 5 的类别（实心点；撤稿处理与主张台账在表中合为一行，图中分开画）与第 5.1 节点名的部分系统（空心点）按两条轴摆放。一条轴是表 5 的最后一列，或描述它们的段落。另一条轴是由谁处理这次变化：可以是系统或 Agent，它们重新标注、合并、重建、重跑或覆盖；也可以是某个人，直接编辑、重新批准或更新整个对象；还可以是研究者，为判断中被触及的部分准入候选。科研知识图谱与实验跟踪器没有画入，因为表 5 没有说明由谁更新它们的记录。只有 ReSearchOS 让被触及的部分保持开放，并经准入处理。

上面的比较记录的是来源所报告的机制，而不是接入或性能；补充说明 4 给出完整综述，其中也包括表 5 背后的检索、记忆与 AI 科学家系统，以及提供了设计若干部分的其他工作。第 2.4 节的一点在这里同样适用：积累还必须容纳纠正，也不能与人的学习混为一谈。因此，生成式 AI 与学习的研究 [15]、人机组合研究 [18] 与反馈循环研究 [17] 都支持分开评价三件事：工件、策略复用，以及研究者学到了什么。这些比较都回答不了的，是这个环境究竟是否必要，而这正是最强反证要追问的问题。

5.2 最强反证与最小形态

什么结果会使本报告大部分内容变得不必要？第 2 节背后的设计文件指明了一项比较：让一个能直接读写文件的强模型配上一页版本化交接，与 ReSearchOS 在足够多的自然研究任务上比较，两边的来源、工具、模型与预算相同。如果模型能在声明的质量界与关键错误界之内恢复当前问题、关键根据、已接受的判断、未决边界与下一步行动，并且总投入更低、在连续两次比较中都能做到，交接即算胜出。交接胜出时，ReSearchOS 应收缩为版本化研究状态、最小上下文路由、研究者权威、溯源和带回滚的安全写回。届时 Dashboard、低价值插件、复杂状态与单独的「操作系统」叙事都可以退役。收缩是这些原则所允许的成功，而不是失败（图 16）。

迄今的证据两方面都有，不足以作出任何裁定。一方面是作者重建的九十天设计史中的四个请求，列于表 6，绘于图 17。已知在每个请求中，续接的线程都缺少上次会话记录没有带过来的条件或去向。它们是迄今仅有的此类案例，所缺的恰恰是一页交接必须补上的内容。这四个请求都不是来源

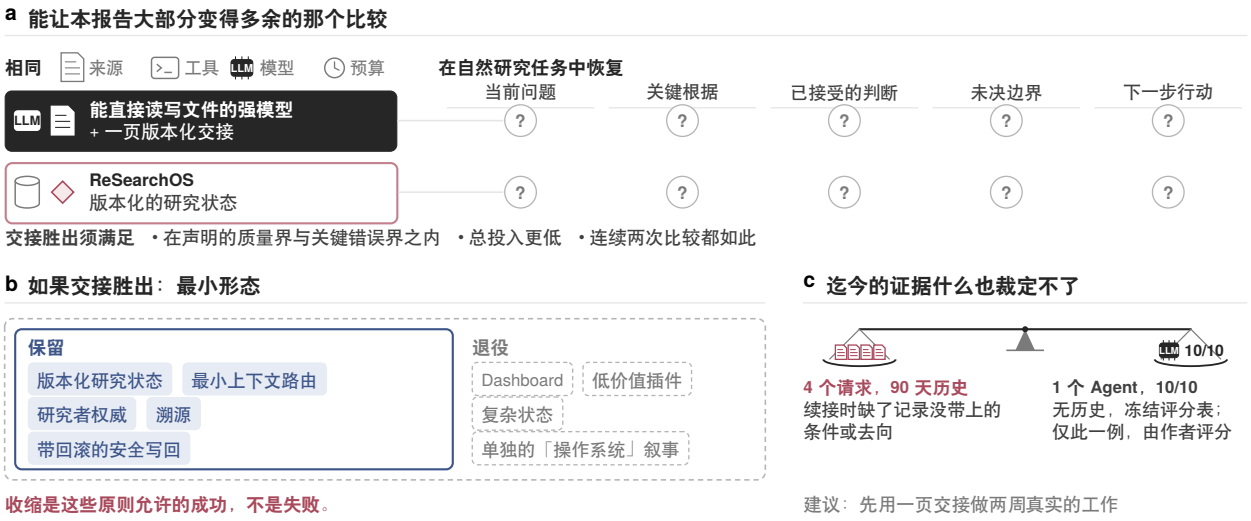


图 16. 一次比较就能让本报告大部分变得多余，而迄今的证据什么也裁定不了。 **a**，这个比较：能直接读写文件的强模型配一页版本化交接，对 ReSearchOS，来源、工具、模型与预算相同，各自按能否在自然研究任务中恢复五项内容计分。交接只有在声明的质量界与关键错误界之内、以更低的总投入、在连续两次比较中都做到，才算胜出。比较尚未进行，所以各格都未计分。**b**，如果交接胜出，ReSearchOS 应收缩为蓝框内的最小形态，框外四项可以退役。**c**，迄今的证据两边都有：重建的九十天历史里的四个请求，以及一个无历史 Agent 得 10/10，这是作者评分的单一案例。

变化，所以本报告建议，这项比较的自然任务中包括一项来源发生变化的任务，就像贯穿例子中的论文那样。

表 6. 四个历史请求及续接时缺了什么

请求	续接时缺了什么	设计回应
选中两张合作者卡片做组合，任务看似完成	看不到产出，也看不到改动该落在哪里	返回带根据的回答，并分别标明每个候选去向；来源可以只读不改
借鉴早期一个记忆相关机制来完善设计说明，同时不把旧结构强行套进新任务	旧机制解决的是什么问题，哪项变了的前提可能让迁移失效	写明拟议对应、变化的前提和最小区分性检查；候选可以被放弃
把一个新的应用兴趣和两位潜在合作者的经验接到已有工作上	资源访问、时间和参与意愿，当时问题和团队都还没定	把这些列为候选问题的可行性条件；经验能解释为何建议某个角色，不代表对方已经同意
为作者承担支持角色的项目整理两份方案文档的理解，并能在以后找到	来源位置、角色边界和尚未解决的问题	把文档和支持角色接到一份可定位的项目说明上；频繁参与不悄然变成牵头

表 6 各行改写自作者本人的记录，是作者事后的重构。它们只作为需求来源的先例，不是恢复出的执行轨迹，也不衡量收益，同样不表明一份维护良好的交接在这些情形下会失败。同一段历史还记录了反复提出的、希望界面少一些杂乱的请求，所记下的设计回应是：在扩展界面之前，先在同一个 episode 上，用能读写文件的强模型加一份维护良好的交接做对照。另一方面，一个拿到参与者包的无历史 Agent 在冻结的评分表上得到 10/10，没有硬失败（附录 B.1）。这是作者评分的单一案例，既不证明仅凭交接就已足够，也不证明它跨任务或跨模型可靠、对参与者有价值。

下一步是协调策略，即环境中在每个决策边界决定检查什么、启动什么、询问什么、什么不去管的那一部分，同时还需要一个任务，用来检验策略能否长期保持正确的状态。与交接基线的比较和第一项使用研究（参与者是作者以外的研究者）都尚未进行，第 2–4 节也没有预设它们的结果。在它



图 17. 四个请求里，续接时缺的都是记录没带过来的条件或去向。 **a**–**d**，依次是表 6 的四个请求，沿用图 1 的画法，每个对象只画一次，用文字标注。左侧是请求时的输入，每条提出的边用编号标出所属阶段：标记 1 是续接工作提出的关系，标记 2 是修订与下一次使用。每格底行写明表 6 的设计回应。虚线对象是提议，不是已接受的内容；橙红色框是挂起的条件。各格是对表 6 的图示。

们完成之前，实际的建议是先用一页交接开展两周的真实工作，只在交接丢失条件或去向之处才安装原型；读者能够安装的公开 starter 回收候选，但不准入候选（第 4.4 节）。

5.3 结论

在 ReSearchOS 看来，科研的连续性是一个状态问题：要紧的是研究要带下去的状态，而不是文件。它用三个相互约束的部分回答这个问题：原则说明一次变化之后什么必须保住，模型定义研究状态及作用于它的三个动作，参考架构与原型再把这些动作分配给记录、内核与客户端。在贯穿会话中，原型恢复了比较，在来源变化时把判断标为待复核，并且只在研究者决定之后才准入收窄的判断；它还记录了研究者对无依据提议的拒绝，收尾时复核仍然开着。这些是起始路径，都可以在终端上检查。

表 7 把每项贡献、本报告给出的证据和尚待证明的部分并列。

表 7. 贡献、证据与尚待证明

贡献	本报告给出的证据	尚待证明
设计原则（第 2 节）	表 6 的四个请求（作为需求来历的先例），以及促成每项原则的文献	能否改善真实研究中的理解与决定；使用研究尚未进行
研究状态模型（第 3 节）	定义，以及在所述假设下论证的两条命题（补充说明 5）	原型没有实现完整的条件组合；两条命题都不证明完成可达
参考架构与原型（第 4 节）	贯穿会话、内核回归测试与附录 B.3 的 CI 运行	未实现：陈述的条件与计划的实验的登记，范围外依赖的等待。未验证：跨客户端集成。未进行：与「强模型加一页交接」的比较

长期收益尚未得到证明。第 5.2 节所说的两项研究都还没有进行；在主张任何价值之前，应先用各自的结果修订模型或原型。清晰的交接已经够用的地方，应选更简单的安排；额外的结构只在帮助研究者注意到条件变化、修订判断、避免重复工作之处才值得保留。这也回到报告开头的回答：要留住的是研究状态，而不是研究文件；来源变化时，哪一部分保留、更新或挂起，由研究者决定，而不是由模型或另一个会话决定。

可用性与伦理。参考工件使用公开文献与虚构例子。第 5.2 节提到的四个设计请求，重建自作者本人记录的九十天历史；这段历史包含合作者与学生的消息和未发表的工作。这段历史只在本地处理，其符号投影不带标识，也没有任何内容对外分发。个人与项目标识、原始私有文档、未发表的技术细节、私有来源和记录图都不随公开报告分发，本文也不宣布数据集或基准发布。当前软件许可为非商业、源码可见，不主张 OSI 认可的开源。工件获取仍受仓库可见性限制。任何参与者研究都需要知情同意、适用的机构审查、数据最小化与退出安排，安全实验则需要与真实外部效果隔离。

声明。 AI 辅助参与了来源发现、起草、翻译、制图与排版；科学主张与投稿元数据由作者负责。参考文献中包含相关的合作研究作为方法来源，不把其结果当作对 ReSearchOS 的验证。本文不报告参与者结果或长期因果收益。

A 对象与要求的生命周期

第 3 节在例子上逐个引入模型的对象，本附录把它们并列在一起。表 8 列出模型维护的每一类对象、它可能处于的状态、它在什么条件下转移，以及让它转移的一方或机制。各方是研究者、系统（即内核与第 4 节的命令）和模型；模型的产出只能以候选身份进入。补充说明 5 给出这些定义的精确表述，并论证第 3.4 节的两条命题。

表 8. 对象、状态与由谁改变

对象	状态	条件	由谁改变
来源或记录的版本	当前版本候选，或已被取代	它是截点前可用且获准的观察中没有已知后继的已记录修订时，就是当前的；只有可见历史确立了的后继才算已知，所以若干未消解的分支可以同时是当前的	系统，在每个决策边界；原型里在下一次 `ros` 命令时
候选	已回收待决定、已准入或已拒绝	只有准入的五个条件同时成立（第 3.1 节）才被准入，第一个条件就是研究者的决定；由研究者拒绝，或在审阅期间依据变化时由准入以过期为由自行拒绝。文件前像不再相符时，被拒绝的是这次写入，而不是候选	由 `ros collect` 或 `ros propose` 回收并冻结其依据；由研究者准入或拒绝；由准入以过期为由自行拒绝
判断	保留、标为待复核或已更新	变化没有触及它的任何根据时保留；它上面的一项要求被触及时标为待复核；只有钉在已变化来源版本上、收窄或替换它的候选被准入，才算更新	系统标记它；研究者的准入更新它。原型中的更新是旧判断旁边的新块；原型既不能替换也不能撤回旧判断，由研究者手动修改或删除（差距 11）
要求（一件维护工作）	未决、已解决或已取消；挂起与等待是未决工作的两种	变化触及它的某个已记录输入或扫描发现它时，它成为未决。在当前版本上的同一组合格回执上每项检查都为 1 时解决；由经记录的决定取消。之后的变化可以再次触及它	系统登记并解决它；研究者取消、延期或回答
挂起的要求	挂在问题上	它有会让它重开的触发条件、至少一项检查为?，且凡不为 1 的检查都是只有研究者能去除的?、各带自己的在册问题时，处于挂起	研究者的回答或触发：确立满足的回答关闭该检查，确立不满足的回答让工作回到未决，触发让它重开
等待的要求	带触发等待	它最近一次记录的转移是等待或延期、带有触发条件，并且被范围外的依赖阻塞或由研究者延期或收窄时，处于等待	研究者的延期或收窄、或范围外的阻塞让它进入等待；触发让它重开

对象	状态	条件	由谁改变
检查	1（通过）、0（不通过）或?（还说不准）；从未求值的检查没有取值	只有合格且当前有效的检查确立满足或不满足时，才取 1 或 0；证据缺失、过期或处在未消解的冲突中时为?；从未求值的检查是阻止完成的未评估工作	依据为机械过程的检查由机制给值；依据为人的检查由研究者的决定及其理由给值
有界问题	待处理、已提出或已回答；因版本变化而作废	检查依据的每个版本只问一次，带有限选项；截点前处于待处理或已提出时为在册问题；因版本变化而作废的问题不算回答过	系统提问；研究者回答
有界目标（原型中为一次会话）	运行中；停止，结果为 `complete`、`complete_with_held`、`deferred`、`budget_stopped` 或 `human_stopped`	开始时冻结规格，并像任何候选一样经过准入；在 Done、某一预算分量耗尽或研究者决定停止时停止；结果在第一次 Stop 时读出	系统冻结规格（原型中为 `ros today`），并判定 Done 与耗尽；研究者可以叫停

这些对象中，承载维护工作的是要求，图 18 画出它的生命周期。挂起与等待让要求留在视野里而不关闭它，因为只有解决和取消才把它移出未决集合；带着挂起或等待的工作，目标仍可以 Done，结果记为 complete_with_held，若没有一项要求得到解决，则记为 deferred。



图 18. 一项要求在记录上关闭之前始终未决；挂起与等待的工作仍然未决。 **a**，要求的某个已记录输入有变化（按修订摘要比较），或初次建立工作、重跑有范围的核对时扫描发现它，它就被触及。只有绑定当前版本的同一组回执上每项检查都为 1，它才得到解决；或由经记录、附理由的决定取消。在此之前，它带有触发条件、凡不为 1 的检查都是只有研究者能去除的?，且每个? 各有一个在册问题（每个依据版本只问一次）时，它处于挂起；最近一次记录的转移是带触发的等待或延期，原因是研究者把它延期或收窄、或冻结范围外的东西阻塞了它时，它处于等待。回答为 0 或触发到来，挂起的工作回到普通的未决；触发到来，等待的工作重开。之后的变化可能再次触及已解决的要求。**b**，0 是确立不满足，工作仍然未决；从未求值的检查没有取值，记为未评估工作。**c**，有界目标中每项未决的已登记要求都已挂起或等待，且没有未评估工作、无法重建的前次规格或仍欠的确认处于未决时，目标 Done；结果在第一次 Stop 时读出。

B 实现与工程记录

第 4 节讲原型做什么，本附录记录它落在哪些提交、命令与测试上。B.1 把第 3 节背后的每条定义对到承载它的命令与回归测试，B.2 说明一次会话怎样代替有界目标，B.3 给出这些提交（与第 4 节同一组）及其测试；全部已披露的差距见补充说明 6。设计义务、形式定义、已实现路径与证据分作不同的主张，所以本附录不会把未实现的机制写成运行能力。凡未注明针对这些提交的实现陈述，都沿用此前验证时的范围，没有在这些提交上重新验收。仓库中保留但不在开发之中的组件（如插件体系）按现状描述，本报告不为它们增加功能。

B.1 从定义到命令

表 9 就第 3 节背后的每条定义回答三件事：它经哪些命令到达研究者，`evals/test_ros_kernel.py` 里哪些内核回归测试演练它，原型还差什么。带编号的定义见补充说明 5。

表 9. 第 3 节的定义在原型中的落点

第 3 节的定义	命令	内核回归测试	原型还差什么
研究状态（第 3.1 节）	`ros state`、`ros context`	`test_state_reports_q_e_j_b_n`；`test_cr2_ros_context_emits_context_receipt_with_egress_service`	研究者陈述的条件与计划的实验不登记（差距 12）；Agent 的直接读取没有回执（差距 7）
准入（第 3.1 节）	`ros collect`、`ros propose`、`ros admit`	`test_hj1_ros_admission_requires_human_decision_record`；`test_proposal_freezes_unpinned_basis_and_admit_rejects_later_change`；`test_admission_does_not_overwrite_an_edit_after_planning`	`test_hj1_…` 检查的是调用渠道被记录，不是要求研究者的决定；Agent 执行的准入与 Dashboard 写入只记录、不阻断（差距 1、2）；不验证任意规则须保持的部分（差距 9）
当前版本候选（定义 5.1）	`ros today`	`test_cr16_exact_replay_adds_no_observation_work_or_question`；`test_today_explains_exact_changes_and_related_requirements`	版本是内容摘要，每个边界每条记录一个，所以定义 5.1 的次序、分支与晚到祖先都不计算。所引回归测试只检验重放与变化发现；没有文件监听，变化要到下一次 `ros` 命令才被看到（差距 34）
被触及与未决的要求（定义 5.2–5.3）	`ros today`	`test_basis_change_opens_then_reconfirm_resolves_and_replay_is_deterministic`；`test_a_b_a_reopens_and_generation_advances_on_new_read_set`	改回原先的内容算回到原先版本并解决复核，不同于定义 5.1 的新修订；不跨多跳传播（差距 34）；新要求的归因不完整（差距 23–25）
三值检查与解决（定义 5.4–5.5）	`ros today`；终端上的 `ros decide … cancel`	`test_a_requirement_closes_on_its_declared_check_bound_to_the_current_version`；`test_cc3_unknown_check_never_resolves_and_stays_apart_from_coverage_unknown`；`test_b8_only_the_terminal_closes_a_requirement_by_decide`	只执行脚本检查，新种类的可执行检查须改动内核（差距 12、25）

第 3 节的定义	命令	内核回归测试	原型还差什么
挂起与等待的工作（定义 5.6–5.10）	`ros today`、`ros answer`、`ros decide … defer`	`test_hj6_undecidable_unknown_is_asked_once_and_waits_outside_open_work`；`test_cc7_defer_channel_controls_completion_and_terminal_decline_does_not_write`	随附规则没有声明以人为依据的检查，所以只有要求的主体或依据无法解析时才走到挂起路径。还没有机制会因范围外依赖而产生等待（差距 17）；过期问题上的挂起可能留存（差距 18）
组合（定义 5.11）	无	无	没有支持通用后继、并行或汇合检查的执行消费者（差距 10）
有界目标、Done 与 Stop（定义 5.20）	`ros today`（含 `--for`）、`ros close`	`test_cc6_episode_spec_is_frozen_and_scan_stays_inside_it`；`test_cc7_special_requirements_exclude_done_in_every_unclosed_state`；`test_cc10_waiting_spends_boundaries_and_an_exhausted_session_stops`	会话只近似有界目标（差距 15、16）；预算没有完整计量（差距 19–22）；比例为零的规则没有执行（差距 18）

每一行从一条定义读到研究者输入的命令、在临时夹具上运行这条命令的回归测试，以及限定这一主张的差距。回归测试确立的是所列夹具上的执行性质；定义是否已完整实现、判断是否正确、研究者是否有所收获，都在它们能说明的范围之外。因此覆盖按定义、按差距来读，从不按测试数量来读。组合是唯一没有命令的定义；有界目标只由会话近似承载（B.2）。

这里还有一项结果，它关系到第 5.2 节的反证，而不是某条定义。一个无历史的本地 Agent 使用参与者包与获准来源，在冻结的 10 分制评分表上得 10/10，没有硬失败；这个单一案例能说明什么、不能说明什么，见第 5.2 节。

B.2 以会话近似有界目标

原型没有单独的目标对象，由一次会话代替第 3.4 节的有界目标。`ros today` 开启会话并冻结它的规格：用途和版本，对象、范围和摘要，规则和保持字段，授权、预算，以及来源和截点。同一天对同一对象再次运行时沿用这份快照并报告差异，已准入的修订只从下一次目标起生效。`--for` 把会话收窄到一项承诺、它的判断、承载这些判断的卡片、它的来源卡与依据卡。对象不存在或不唯一时不开会话；范围为空或没有快照时，不能报 complete 或 complete_with_held（差距 16）。

预算覆盖边界、提问与重开，随规格冻结。每个边界计入所属会话，即使某个边界上只剩等待在继续，也照样计入，所以预算数的是边界，不是经过的时间。预算耗尽时停止提问、派发和后续边界，但不停止要求维护与重开记录；此时未完成就收尾，结果为 budget_stopped，除非旧的挂起问题提供豁免。差距 19–22 记录计量与核算的缺口。

等待只有在研究者带着尚未到期的触发把它放进去时，才获得完成豁免：经研究者本人的渠道、带着触发条件延期或收窄，或在终端确认 `ros decide … defer`。所以终端确认的延期确实会产生等待，而还没有机制会因范围外依赖而产生等待（差距 17）。不在终端作出的延期（记为 `invoker_asserted`）、未经确认的旧延期、执行方延期、范围内依赖，以及没有触发或已到期的等待，都不能获得豁免。有几种情形始终阻止完成：前次规格不可重建，break-glass 判断块或字段确认，范围内的未评估工作，以及已派出却无可用结果的任务（含失败且副作用未知的）。派出前失败则不阻止。不过由回答触发的挂起项仍可能依赖无效的旧问题（差距 18）。明确放弃时如实记下副作用未知，不出回执，但允许收尾。

收尾按要求而非输入计数，只有范围内的未评估工作影响完成。未完成就提前收尾，记为 `human_stopped`；全部问题不论范围都会列出，不可核实的回执也不能排除范围内既有的要求。每个边界都冻结足够的输入，让后一个边界能区分前次规格下漏检的要求与由变化引起的要求。冷启动记为 initial；前次规格缺失也记为 initial，但这时漏检状态未知，系统登记唯一一条「规格不可重建」的要求，若它已关闭则重开。新要求按范围、来源与版本变化的归因不完整，语义缓存也尚未实现（差距 23–25）。

B.3 代码与测试

第 4 节与本附录描述的是公开 starter 的提交 f541cb6d161a 与私有原型的提交 4a9f696a6ebc。在 CI 中，公开 starter 的 661 项 Python 回归测试在 CPython 3.10 与 3.13 上通过，私有版的默认回归集在 CPython 3.11 上通过；不在开发之中的组件，其测试不在默认集内。这些是对所述提交的软件检查，既不构成人工验收，也不证明科学有效性。

参考文献

[1] MacLean, Allan; Young, Richard M.; Bellotti, Victoria M. E. et al. Questions, Options, and Criteria: Elements of Design Space Analysis. Human–Computer Interaction 6(3–4), 201–250, 1991. https://doi.org/10.1207/s15327051hci0603&4_2.

[2] Singh, Jatinder; Cobbe, Jennifer; Norval, Chris. Decision Provenance: Harnessing Data Flow for Accountable Systems. IEEE Access 7:6562–6574, 2019. https://doi.org/10.1109/ACCESS.2018.2887201.

[3] Buçinca, Zana; Malaya, Maja Barbara; Gajos, Krzysztof Z. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. Proceedings of the ACM on Human-Computer Interaction, 2021. https://doi.org/10.1145/3449287.

[4] Ma, Yiping; Li, Xinjin; Hu, Shiyu et al. Trustworthy AI in education: Framework, cases, and governance strategies. Innovation and Emerging Technologies, 2025. https://doi.org/10.1142/S2737599425500264.

[5] Clark, Herbert H.; Brennan, Susan E. Grounding in communication. Perspectives on Socially Shared Cognition, 1991. https://doi.org/10.1037/10096-006.

[6] Simon, Herbert A. A Behavioral Model of Rational Choice. The Quarterly Journal of Economics 69(1):99–118, 1955. https://doi.org/10.2307/1884852.

[7] Malone, Thomas W.; Crowston, Kevin. The Interdisciplinary Study of Coordination. ACM Computing Surveys 26(1), 87–119, 1994. https://ccs.mit.edu/papers/CCSWP157.html.

[8] Lappas, Theodoros; Liu, Kun; Terzi, Evimaria. Finding a team of experts in social networks. Proceedings of KDD 2009, pp. 467–476. https://cs-people.bu.edu/evimaria/cs591/lappas09finding.pdf.

[9] Argyris, Chris. Single-Loop and Double-Loop Models in Research on Decision Making. Administrative Science Quarterly, 1976. https://doi.org/10.2307/2391848.

[10] Kleppmann, Martin; Wiggins, Adam; van Hardenberg, Peter et al. Local-first software: you own your data, in spite of the cloud. Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software. https://doi.org/10.1145/3359591.3359737.

[11] Wilson, Greg; Bryan, Jennifer; Cranston, Karen et al. Good enough practices in scientific computing. PLOS Computational Biology, 2017. https://doi.org/10.1371/journal.pcbi.1005510.

[12] Parnas, D. L. On the criteria to be used in decomposing systems into modules. Communications of the ACM, 1972. https://doi.org/10.1145/361598.361623.

[13] Walsh, James P.; Ungson, Gerardo Rivera. Organizational Memory. Academy of Management Review, 1991. https://doi.org/10.5465/amr.1991.4278992.

[14] March, James G. Exploration and Exploitation in Organizational Learning. Organization Science 2(1):71–87, 1991. https://doi.org/10.1287/orsc.2.1.71.

[15] Yan, Lixiang; Greiff, Samuel; Teuber, Ziwen et al. Promises and challenges of generative artificial intelligence for human learning. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02004-5.

[16] Flavell, John H. Metacognition and cognitive monitoring: A new area of cognitive-developmental inquiry. American Psychologist, 1979. https://doi.org/10.1037/0003-066X.34.10.906.

[17] Glickman, Moshe; Sharot, Tali. How human–AI feedback loops alter human perceptual, emotional and social judgements. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02077-2.

[18] Vaccaro, Michelle; Almaatouq, Abdullah; Malone, Thomas. When combinations of humans and AI are useful: A systematic review and meta-analysis. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-02024-1.

[19] Mitchell, Margaret; Wu, Simone; Zaldivar, Andrew et al. Model Cards for Model Reporting. ACM FAT* 2019. https://doi.org/10.1145/3287560.3287596.

[20] Ribeiro, Marco Tulio; Wu, Tongshuang; Guestrin, Carlos et al. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. ACL 2020:4902–4912. https://doi.org/10.18653/v1/2020.acl-main.442.

[21] Hu, Shiyu; Zhang, Dailing; Wu, Meiqi et al. A Multi-modal Global Instance Tracking Benchmark (MGIT): Better Locating Target in Complex Spatio-temporal and Causal Relationship. Advances in Neural Information Processing Systems 36, 2023. https://proceedings.nips.cc/paper_files/paper/2023/hash/4ea14e6090343523ddcd5d3ca449695f-Abstract-Datasets_and_Benchmarks.html.

[22] Li, Xuchen; Li, Xuzhao; Hu, Shiyu et al. CausalStep: A Benchmark for Explicit Stepwise Causal Reasoning in Videos. AAAI 40(8):6530–6538, 2026. https://doi.org/10.1609/aaai.v40i8.37582.

[23] Li, Xuchen; Li, Xuzhao; Hu, Shiyu et al. Select Less, Reason More: Prioritizing Evidence Purity for Video Reasoning. CVPR 2026. https://openaccess.thecvf.com/content/CVPR2026/html/Li_Select_Less_Reason_More_Prioritizing_Evidence_Purity_for_Video_Reasoning_CVPR_2026_paper.html.

[24] Hu, Shiyu; Li, Xuchen; Li, Xuzhao et al. FIOVA: A Multi-Annotator Benchmark for Human-Aligned Video Captioning. arXiv preprint, 2025. arXiv:2410.15270. https://arxiv.org/abs/2410.15270.

[25] Abadi, Martin; Lamport, Leslie. The Existence of Refinement Mappings. Theoretical Computer Science 82(2), pp. 253–284, 1991. https://research.google/pubs/the-existence-of-refinement-mappings-2/.

[26] Horvitz, Eric. Principles of Mixed-Initiative User Interfaces. Proceedings of CHI 1999. https://doi.org/10.1145/302979.303030.

[27] Madaan, Aman; Tandon, Niket; Gupta, Prakhar et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. https://doi.org/10.52202/075280-2019.

[28] Shinn, Noah; Cassano, Federico; Gopinath, Ashwin et al. Reflexion: language agents with verbal reinforcement learning. NeurIPS 2023. https://doi.org/10.52202/075280-0377.

[29] Hollan, James; Hutchins, Edwin; Kirsh, David. Distributed Cognition: Toward a New Foundation for Human-Computer Interaction Research. ACM Transactions on Computer-Human Interaction, 2000. https://doi.org/10.1145/353485.353487.

[30] Cheney, James; Ahmed, Amal; Acar, Umut A. Provenance as dependency analysis. Mathematical Structures in Computer Science 21(6):1301–1337, 2011. https://doi.org/10.1017/S0960129511000211.

[31] Heckel, Reiko. Graph Transformation in a Nutshell. Electronic Notes in Theoretical Computer Science 148(1):187–198, 2006. https://doi.org/10.1016/j.entcs.2005.12.018.

[32] Kung, H. T.; Robinson, John T. On Optimistic Methods for Concurrency Control. ACM Transactions on Database Systems 6(2), 1981. https://doi.org/10.1145/319566.319567.

[33] Parker Jr., D. Stott; Popek, Gerald J.; Rudisin, Gerard et al. Detection of Mutual Inconsistency in Distributed Systems. IEEE Transactions on Software Engineering SE-9(3), 1983. https://doi.org/10.1109/TSE.1983.236733.

[34] Doyle, Jon. A Truth Maintenance System. Artificial Intelligence 12(3), 1979. https://doi.org/10.1016/0004-3702(79)90008-0.

[35] McAllester, David. Truth Maintenance. Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90), 1109–1116, 1990. https://cdn.aaai.org/AAAI/1990/AAAI90-164.pdf.

[36] de Kleer, Johan. An assumption-based TMS. Artificial Intelligence 28(2), 127–162, 1986. https://doi.org/10.1016/0004-3702(86)90080-9.

[37] Alchourrón, Carlos E.; Gärdenfors, Peter; Makinson, David. On the logic of theory change: Partial meet contraction and revision functions. Journal of Symbolic Logic 50(2), 510–530, 1985. https://doi.org/10.2307/2274239.

[38] Mokhov, Andrey; Mitchell, Neil; Peyton Jones, Simon. Build Systems à la Carte. Proceedings of the ACM on Programming Languages 2(ICFP), 2018. https://doi.org/10.1145/3236774.

[39] Rasmussen, Preston; Paliychuk, Pavlo; Beauvais, Travis et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv preprint, 2025. arXiv:2501.13956. https://arxiv.org/abs/2501.13956.

[40] Gazit, Tiferet. Building an agentic memory system for GitHub Copilot. Engineering report, The GitHub Blog, 2026. https://github.blog/ai-and-ml/github-copilot/building-an-agentic-memory-system-for-github-copilot/.

[41] Wang, Yiqi; Liu, Jiaqi; Zhang, Jiaqi et al. PACMI: Provenance-Aware Cascading Memory Invalidation for Long-Term LLM Agents. arXiv preprint, 2026. arXiv:2610.05732. https://arxiv.org/abs/2610.05732.

[42] Qi, Yijiashun; Xu, Xiang; Li, Yuxuan. When Not to Write Memory: Governing False Promotion from Correlated Agent Traces. arXiv preprint, 2026. arXiv:2607.02579. https://arxiv.org/abs/2607.02579.

[43] Schmidgall, Samuel; Su, Yusheng; Wang, Ze et al. Agent Laboratory: Using LLM Agents as Research Assistants. Findings of EMNLP 2025:5977–6043. https://doi.org/10.18653/v1/2025.findings-emnlp.320.

[44] Mitchener, Ludovico; Yiu, Angela; Chang, Benjamin et al. Kosmos: An AI Scientist for Autonomous Discovery. arXiv preprint, 2025. https://arxiv.org/abs/2511.02824.

[45] Jaradeh, Mohamad Yaser; Oelen, Allard; Farfar, Kheir Eddine et al. Open Research Knowledge Graph: Next Generation Infrastructure for Semantic Scholarly Knowledge. Proceedings of the 10th International Conference on Knowledge Capture (K-CAP 2019). https://doi.org/10.1145/3360901.3364435.

[46] IBM. Link validity. Software documentation, IBM Engineering Lifecycle Management 7.0.3, 2026. https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/lifecycle-management/7.0.3?topic=management-link-validity.

[47] GitHub. About protected branches. Software documentation, GitHub Docs, 2026. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches.

[48] Brooker, Joanne; Synnot, Anneliese; McDonald, Steve et al. Guidance for the production and publication of Cochrane living systematic reviews: Cochrane Reviews in living mode. Cochrane guidance, Version 1.0, December 2019. https://resources.cochrane.org/sites/resources.cochrane.org/files/uploads/inline-files/Transform/201912_LSR_Revised_Guidance.pdf.

[49] Cochrane. Cochrane strengthens systems to manage retracted publications in its published reviews. News release, Cochrane, 2026. https://www.cochrane.org/about-us/news/cochrane-strengthens-systems-manage-retracted-publications-its-published-reviews.

[50] Magee, Liam. Geist in the Machine: Simulating Recognition and Inner Dialogue in AI-Mediated Teaching and Research. arXiv preprint, 2026. arXiv:2603.10450. https://arxiv.org/abs/2603.10450.

[51] Shao, Chenyang; Huang, Dehao; Li, Yu et al. OmniScientist: Toward a Co-evolving Ecosystem of Human and AI Scientists. arXiv preprint, 2025. https://arxiv.org/abs/2511.16931.

[52] Jiang, Tao; Navet, Benjamin; Taghzouti, Yousouf et al. Magnolia: A Persistent-Memory Agent Framework for Computational Chemistry. arXiv preprint, 2026. arXiv:2610.27230. https://arxiv.org/abs/2610.27230.

[53] Xia, Yihan; Wang, Taotao. ResearchLoop: An Evidence-Gated Control Plane for AI-Assisted Research. arXiv preprint, 2026. arXiv:2605.28282. https://arxiv.org/abs/2605.28282.

[54] Engelbart, Douglas C. Augmenting Human Intellect: A Conceptual Framework. SRI Summary Report AFOSR-3223, 1962. https://www.dougengelbart.org/content/view/138/.

[55] Amershi, Saleema; Weld, Dan; Vorvoreanu, Mihaela et al. Guidelines for Human-AI Interaction. Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. https://doi.org/10.1145/3290605.3300233.

[56] Wu, Tongshuang; Terry, Michael; Cai, Carrie Jun. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. ACM CHI 2022. https://doi.org/10.1145/3491102.3517582.

[57] Acar, Umut A.; Blume, Matthias; Donham, Jacob. A Consistent Semantics of Self-adjusting Computation. ESOP 2007, 458–474. https://doi.org/10.1007/978-3-540-71316-6_31.

[58] Jiang, Yucheng; Shao, Yijia; Ma, Dekun et al. Into the Unknown Unknowns: Engaged Human Learning through Participation in Language Model Agent Conversations. EMNLP 2024. https://doi.org/10.18653/v1/2024.emnlp-main.554.

[59] Litt, Geoffrey; Schoening, Max; Shen, Paul et al. Potluck: Dynamic documents as personal software. Ink & Switch research essay, 2022. https://www.inkandswitch.com/potluck/.

[60] Chan, Joel; Akamatsu, Matthew; Vargas, David et al. Steps Towards an Infrastructure for Scholarly Synthesis. arXiv preprint, 2024. arXiv:2407.20666. https://arxiv.org/abs/2407.20666.

[61] Xu, Wujiang; Liang, Zujie; Mei, Kai et al. A-Mem: Agentic Memory for LLM Agents. NeurIPS 2025. https://doi.org/10.52202/085713-0593.

[62] Young, Justin. Effective harnesses for long-running agents. Engineering report, 2025. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents.

[63] Rajasekaran, Prithvi. Harness design for long-running application development. Anthropic Engineering report, 2026. https://www.anthropic.com/engineering/harness-design-long-running-apps.

[64] Cemri, Mert; Pan, Melissa Z; Yang, Shuyi et al. Why Do Multi-Agent LLM Systems Fail? NeurIPS 2025 Datasets and Benchmarks Track. https://doi.org/10.52202/085713-4082.

[65] Anthropic. Claude Code release v2.1.277. Software release notes, 2026. https://github.com/anthropics/claude-code/releases/tag/v2.1.277.

[66] Wei, Jason; Wang, Xuezhi; Schuurmans, Dale et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. https://doi.org/10.52202/068431-1800.

[67] Wang, Xuezhi; Wei, Jason; Schuurmans, Dale et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR 2023. https://openreview.net/forum?id=1PL1NIMMrw.

[68] Yao, Shunyu; Yu, Dian; Zhao, Jeffrey et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 2023. https://doi.org/10.52202/075280-0517.

[69] Yao, Shunyu; Zhao, Jeffrey; Yu, Dian et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://openreview.net/forum?id=WE_vluYUL-X.

[70] Schick, Timo; Dwivedi-Yu, Jane; Dessi, Roberto et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. https://doi.org/10.52202/075280-2997.

[71] Patil, Shishir G.; Zhang, Tianjun; Wang, Xin et al. Gorilla: Large Language Model Connected with Massive APIs. NeurIPS 2024. https://doi.org/10.52202/079017-4020.

[72] Wu, Qingyun; Bansal, Gagan; Zhang, Jieyu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. COLM 2024. https://openreview.net/forum?id=BAakY1hNKS.

[73] Hong, Sirui; Zhuge, Mingchen; Chen, Jonathan et al. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html.

[74] Qian, Chen; Liu, Wei; Liu, Hongzhang et al. ChatDev: Communicative Agents for Software Development. ACL 2024. https://doi.org/10.18653/v1/2024.acl-long.810.

[75] Gao, Dawei; Li, Zitao; Pan, Xuchen et al. AgentScope: A Flexible yet Robust Multi-Agent Platform. arXiv preprint, 2024. arXiv:2402.14034. https://arxiv.org/abs/2402.14034.

[76] Wang, Guanzhi; Xie, Yuqi; Jiang, Yunfan et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. Transactions on Machine Learning Research, 2024. https://openreview.net/forum?id=ehfRiF0R3a.

[77] Bommasani, Rishi; Hudson, Drew A.; Adeli, Ehsan et al. On the Opportunities and Risks of Foundation Models. arXiv preprint, 2022. arXiv:2108.07258. https://arxiv.org/abs/2108.07258.

[78] Karpukhin, Vladimir; Oguz, Barlas; Min, Sewon et al. Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020. https://doi.org/10.18653/v1/2020.emnlp-main.550.

[79] Guu, Kelvin; Lee, Kenton; Tung, Zora et al. Retrieval Augmented Language Model Pre-Training. ICML 2020; PMLR 119:3929–3938. https://proceedings.mlr.press/v119/guu20a.html.

[80] Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[81] Izacard, Gautier; Grave, Edouard. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. EACL 2021. https://doi.org/10.18653/v1/2021.eacl-main.74.

[82] Borgeaud, Sebastian; Mensch, Arthur; Hoffmann, Jordan et al. Improving Language Models by Retrieving from Trillions of Tokens. ICML 2022; PMLR 162:2206–2240. https://proceedings.mlr.press/v162/borgeaud22a.html.

[83] Izacard, Gautier; Lewis, Patrick; Lomeli, Maria et al. Atlas: Few-shot Learning with Retrieval Augmented Language Models. Journal of Machine Learning Research 24(251):1–43, 2023. https://jmlr.org/papers/v24/23-0037.html.

[84] Gao, Yunfan; Xiong, Yun; Gao, Xinyu et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint, 2024. arXiv:2312.10997. https://arxiv.org/abs/2312.10997.

[85] Trivedi, Harsh; Balasubramanian, Niranjan; Khot, Tushar et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. ACL 2023. https://doi.org/10.18653/v1/2023.acl-long.557.

[86] Asai, Akari; Wu, Zeqiu; Wang, Yizhong et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html.

[87] Zhang, Tianjun; Patil, Shishir G.; Jain, Naman et al. RAFT: Adapting Language Model to Domain Specific RAG. COLM 2024. https://openreview.net/forum?id=rzQGHXNReU.

[88] Chen, Tong; Wang, Hongwei; Chen, Sihao et al. Dense X Retrieval: What Retrieval Granularity Should We Use? EMNLP 2024. https://doi.org/10.18653/v1/2024.emnlp-main.845.

[89] Edge, Darren; Trinh, Ha; Cheng, Newman et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv preprint, 2025. arXiv:2404.16130. https://arxiv.org/abs/2404.16130.

[90] Gutiérrez, Bernal J.; Shu, Yiheng; Gu, Yu et al. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. NeurIPS 2024. https://doi.org/10.52202/079017-1902.

[91] Packer, Charles; Wooders, Sarah; Lin, Kevin et al. MemGPT: Towards LLMs as Operating Systems. arXiv preprint, 2024. arXiv:2310.08560. https://arxiv.org/abs/2310.08560.

[92] Zhong, Wanjun; Guo, Lianghong; Gao, Qiqi et al. MemoryBank: Enhancing Large Language Models with Long-Term Memory. AAAI 38(17):19724–19731, 2024. https://doi.org/10.1609/aaai.v38i17.29946.

[93] Park, Joon Sung; O'Brien, Joseph; Cai, Carrie Jun et al. Generative Agents: Interactive Simulacra of Human Behavior. ACM UIST 2023. https://doi.org/10.1145/3586183.3606763.

[94] Chhikara, Prateek; Khant, Dev; Aryan, Saket et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. ECAI 2025:2993–3000. https://doi.org/10.3233/FAIA251160.

[95] Letta. MemFS. Software documentation, Letta Docs, 2026. https://docs.letta.com/concepts/memfs.

[96] Letta. Letta Code v0.33.0. Software release notes, GitHub, 2026. https://github.com/letta-ai/letta-code/releases/tag/v0.33.0.

[97] Sumers, Theodore R.; Yao, Shunyu; Narasimhan, Karthik et al. Cognitive Architectures for Language Agents. Transactions on Machine Learning Research, 2024. https://openreview.net/forum?id=1i6ZCyf1QJ.

[98] Hu, Yuyang; Liu, Shichun; Yue, Yanwei et al. Memory in the Age of AI Agents. arXiv preprint, 2026. arXiv:2512.13564. https://arxiv.org/abs/2512.13564.

[99] Du, Pengfei. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers. arXiv preprint, 2026. arXiv:2603.07670. https://arxiv.org/abs/2603.07670.

[100] Wu, Ming; Zhu, Pengyuan. Agent Zero Memory: Provenance-Aware Long-Term Memory for LLM Agents. arXiv preprint, 2026. arXiv:2608.29606. https://arxiv.org/abs/2608.29606.

[101] Wilkinson, Mark D.; Dumontier, Michel; Aalbersberg, IJsbrand Jan et al. The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 2016. https://doi.org/10.1038/sdata.2016.18.

[102] Soiland-Reyes, Stian; Sefton, Peter; Crosas, Mercè et al. Packaging research artefacts with RO-Crate. Data Science 5(2):97–138, 2022. https://doi.org/10.3233/DS-210053.

[103] RO-Crate Community. RO-Crate Metadata Specification 1.3.0. Specification, 2026. doi:10.5281/zenodo.20720080. https://www.researchobject.org/ro-crate/specification/1.3/.

[104] Greshake, Kai; Abdelnabi, Sahar; Mishra, Shailesh et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. ACM AISec 2023 Workshop. https://doi.org/10.1145/3605764.3623985.

[105] Zhan, Qiusi; Liang, Zhixiang; Ying, Zifan et al. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL 2024. https://doi.org/10.18653/v1/2024.findings-acl.624.

[106] Debenedetti, Edoardo; Zhang, Jie; Balunovic, Mislav et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-2636.

[107] Cerruti, Tommaso; Okamoto, Mika; Erol, Ansel Kaplan. Agent Memory Is a Surface for Endogenous Authorization Laundering. arXiv preprint, 2026. arXiv:2609.01836. https://arxiv.org/abs/2609.01836.

[108] Hu, Shiyu; Zhao, Xin; Huang, Lianghua et al. Global Instance Tracking: Locating Target More Like Humans. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2023. https://doi.org/10.1109/TPAMI.2022.3153312.

[109] Zhao, Xin; Hu, Shiyu; Wang, Yipei et al. BioDrone: A Bionic Drone-Based Single Object Tracking Benchmark for Robust Vision. International Journal of Computer Vision, 2024. https://doi.org/10.1007/s11263-023-01937-0.

[110] Hu, Shiyu; Zhao, Xin; Huang, Kaiqi. SOTVerse: A User-Defined Task Space of Single Object Tracking. International Journal of Computer Vision, 2024. https://doi.org/10.1007/s11263-023-01908-5.

[111] Kramer, Stefan; Cerrato, Mattia; Brugger, Jannis et al. Automated Scientific Discovery: From Equation Discovery to Autonomous Discovery Systems. Machine Learning 115:109, 2026. https://doi.org/10.1007/s10994-025-06955-2.

[112] Bran, Andres M.; Cox, Sam; Schilter, Oliver et al. Augmenting large language models with chemistry tools. Nature Machine Intelligence 6:525–535, 2024. https://doi.org/10.1038/s42256-024-00832-8.

[113] Boiko, Daniil A.; MacKnight, Robert; Kline, Ben et al. Autonomous chemical research with large language models. Nature, 2023. https://doi.org/10.1038/s41586-023-06792-0.

[114] Lu, Chris; Lu, Cong; Lange, Robert Tjarko et al. Towards end-to-end automation of AI research. Nature 651:914–919, 2026. https://doi.org/10.1038/s41586-026-10265-5.

[115] Gottweis, Juraj; Weng, Wei-Hung; Daryin, Alexander et al. Accelerating scientific discovery with Co-Scientist. Nature 655:487–496, 2026. https://doi.org/10.1038/s41586-026-10644-y.

[116] Ghafarollahi, Alireza; Buehler, Markus J. SciAgents: Automating Scientific Discovery Through Bioinspired Multi-Agent Intelligent Graph Reasoning. Advanced Materials 37(22):2413523, 2025. https://doi.org/10.1002/adma.202413523.

[117] Ghareeb, Ali E.; Chang, Benjamin; Mitchener, Ludovico et al. A multi-agent system for automating scientific discovery. Nature 655:497–505, 2026. https://doi.org/10.1038/s41586-026-10652-y.

[118] Luo, Jixiang. XScientist: A Git-Like Research Protocol for Long-Running Autonomous Scientific Discovery. arXiv preprint, 2026. arXiv:2607.12301. https://arxiv.org/abs/2607.12301.

[119] Ma, Yiping; Hu, Shiyu; Li, Xuchen et al. When LLMs Learn to be Students: The SOEI Framework for Modeling and Evaluating Virtual Student Agents in Educational Interaction. arXiv preprint, 2025. arXiv:2410.15701. https://arxiv.org/abs/2410.15701.

[120] Ma, Yiping; Hu, Shiyu; Zhu, Buyuan et al. EduVerse: A User-Defined Multi-Agent Simulation Space for Education Scenario. arXiv preprint, 2025. arXiv:2510.05650. https://arxiv.org/abs/2510.05650.

[121] Zhu, Buyuan; Hu, Shiyu; Ma, Yiping et al. EduPersona: Benchmarking Subjective Ability Boundaries of Virtual Student Agents. arXiv preprint, 2025. arXiv:2510.04648. https://arxiv.org/abs/2510.04648.

[122] Liu, Xiao; Yu, Hao; Zhang, Hanchen et al. AgentBench: Evaluating LLMs as Agents. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html.

[123] Ma, Chang; Zhang, Junlei; Zhu, Zhihao et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-2365.

[124] Min, Sewon; Krishna, Kalpesh; Lyu, Xinxi et al. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. EMNLP 2023:12076–12100. https://doi.org/10.18653/v1/2023.emnlp-main.741.

[125] Gao, Tianyu; Yen, Howard; Yu, Jiatong et al. Enabling Large Language Models to Generate Text with Citations. EMNLP 2023:6465–6488. https://doi.org/10.18653/v1/2023.emnlp-main.398.

[126] Es, Shahul; James, Jithin; Espinosa Anke, Luis et al. RAGAs: Automated Evaluation of Retrieval Augmented Generation. EACL 2024 System Demonstrations. https://doi.org/10.18653/v1/2024.eacl-demo.16.

[127] Chen, Jiawei; Lin, Hongyu; Han, Xianpei et al. Benchmarking Large Language Models in Retrieval-Augmented Generation. AAAI 38(16):17754–17762, 2024. https://doi.org/10.1609/aaai.v38i16.29728.

[128] Yang, Xiao; Sun, Kai; Xin, Hao et al. CRAG - Comprehensive RAG Benchmark. NeurIPS 2024 Datasets and Benchmarks Track. https://doi.org/10.52202/079017-0335.

[129] Zheng, Lianmin; Chiang, Wei-Lin; Sheng, Ying et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks Track. https://doi.org/10.52202/075280-2020.

[130] Li, Xuzhao; Li, Xuchen; Hu, Shiyu et al. VerifyBench: A Systematic Benchmark for Evaluating Reasoning Verifiers Across Domains. AAAI 40(38), 2026. https://doi.org/10.1609/aaai.v40i38.40448.

[131] Burton, Jason W.; Lopez-Lopez, Ezequiel; Hechtlinger, Shahar et al. How large language models can reshape collective intelligence. Nature Human Behaviour, 2024. https://doi.org/10.1038/s41562-024-01959-9.

[132] Li, Zhiyu; Song, Shichao; Wang, Hanyu et al. MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models. arXiv preprint, 2025. https://arxiv.org/abs/2505.22101.

[133] Asai, Akari; He, Jacqueline; Shao, Rulin et al. Synthesizing scientific literature with retrieval-augmented language models. Nature, 2026. https://doi.org/10.1038/s41586-025-10072-4.

[134] Skarlinski, Michael D.; Cox, Sam; Laurent, Jon M. et al. Language agents achieve superhuman synthesis of scientific knowledge. arXiv preprint, 2024. https://arxiv.org/abs/2409.13740.

[135] Villaescusa-Navarro, Francisco; Bolliet, Boris; Villanueva-Domingo, Pablo et al. The Denario project: Deep knowledge AI agents for scientific discovery. arXiv preprint, 2025. https://arxiv.org/abs/2510.26887.

[136] Lyu, Yougang; Zhang, Xi; Yi, Xinhao et al. EvoScientist: Towards Multi-Agent Evolving AI Scientists for End-to-End Scientific Discovery. arXiv preprint, 2026. https://arxiv.org/abs/2603.08127.

[137] Marshall, Catherine C.; Shipman III, Frank M.; Coombs, James H. VIKI: Spatial Hypertext Supporting Emergent Structure. Proceedings of ECHT '94, pp. 13–23, 1994. https://people.engr.tamu.edu/shipman/abstracts/echt94-abstract.html.